Practical Cryptographic Systems

Protocols I

Instructor: Matthew Green

News?

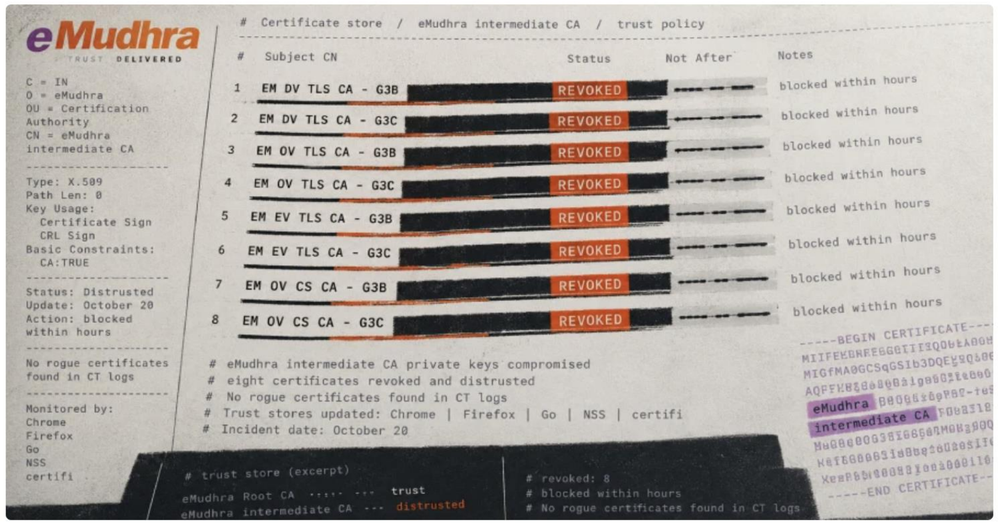
The Hacker News

Get the Latest News

Eight eMudhra CA Keys Compromised, Triggering Emergency Fixes Across Browsers and Software

Swati Khandelwal Oct 20, 2026

Web Security / Browser Security



The private keys of eight eMudhra intermediate certificate authority (CA) certificates are compromised, Mozilla and Google's Chrome Root Program confirms on October 20.

Top Stories This Week



Microsoft Patches Record 1,413 Flaws, Including Exploited Windows Shell Bug



Langflow's Critical Authenticated RCE Flaw Targeted in 1,423 Honeypot Exploit Attempts



AI-Found Linux Kernel Flaw Gives Local Users Root, and Can Break a Guest Out to the Host



curl Patches HTTP/2 Upload Flaw After PoC Demonstrates Client-Side Code Execution



GitLab Patches CVSS 10.0 Flaw That Could Let Unauthenticated Users Run Server Code



Attackers Heat French Supermarket Fridges Up to 50°C by Hijacking Danfoss

News?

Intermediate	Type	Private key (d)	Key status	Cert status
EM DV TLS CA - G3B	TLS (DV)	156	Public / trivially recovered	Revoked Oct 18; blocked in Chrome, Firefox, Apple, Windows
EM DV TLS CA - G3C	TLS (DV)	230	Public / trivially recovered	Revoked; blocked
EM OV TLS CA - G3B	TLS (OV)	155	Public / trivially recovered	Revoked; blocked
EM OV TLS CA - G3C	TLS (OV)	4	Public / trivially recovered (Flüpke posted it on GitHub)	Revoked; blocked
EM EV TLS CA - G3B	TLS (EV)	115	Public / trivially recovered	Revoked; blocked
EM EV TLS CA - G3C	TLS (EV)	56	Public / trivially recovered	Revoked; blocked
EM OV CS CA - G3B	Code signing	66	Public / trivially recovered	Revoked (not covered by Chrome's CRLSet, which only hit the six TLS CAs)
EM OV CS CA - G3C	Code signing	237	Public / trivially recovered	Revoked

Review

- Last time:
  - Public-key encryption
  - RSA, Lattices (ML-KEM, ML-DSA) all at the most superficial level

This time

- Protocols
  - So far we've been exhaustively discussing primitives
  - Today we're going to discuss the protocols that use those protocols

Protocols (definition)

- Definition
  - “A set of rules or procedures for transmitting data between electronic devices, such as computers”
  - “A security protocol (cryptographic protocol or encryption protocol) is an abstract or concrete protocol that performs a security-related function and applies cryptographic methods”

TLS today

Secure channels
TLS protects traffic between a client and a server.

Current protocol
TLS 1.3, originally standardized in 2018.

Compatibility
TLS 1.2 remains in use. SSL 3 and TLS 1.0/1.1 are obsolete.

Beyond web browsing
TLS also secures mail, APIs, and many other applications.

This lecture uses historical attacks to explain modern design.

How secure is TLS?

Protocol failures
BEAST, CRIME, FREAK, and Logjam exposed design or negotiation weaknesses.

Implementation failures
Heartbleed exposed process memory in affected OpenSSL versions.

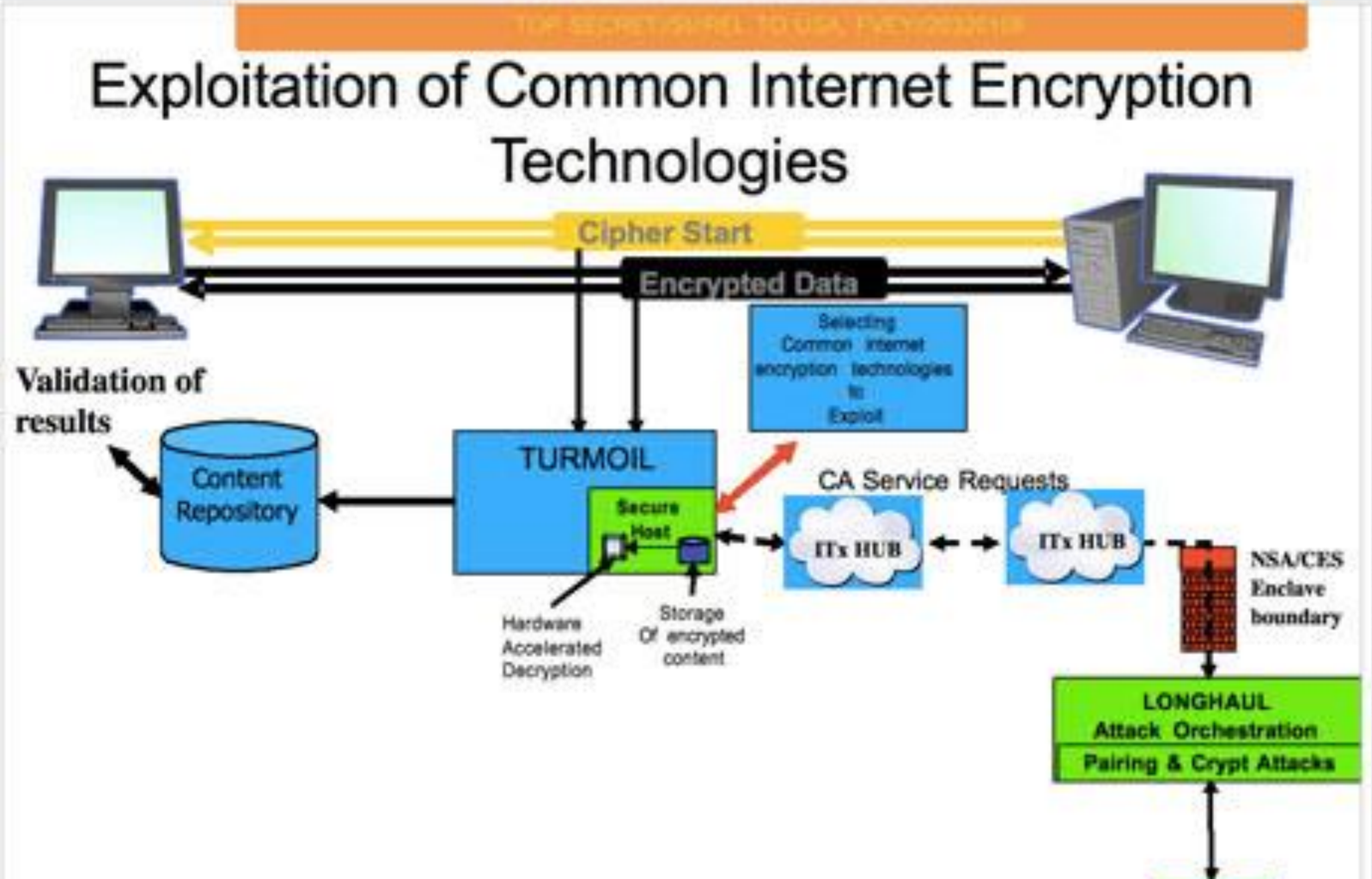
Attack cost varies
Some attacks need many controlled observations. Others permit broad probing.

The lesson
A sound protocol, safe configuration, and correct implementation all matter.

The following case studies concern older protocol versions and software.

How secure is TLS?

But not all attacks…



A brief history

- **SSL 3 / TLS 1.0 (1996–1999)**
  - Predictable IVs, renegotiation attacks and more!

Hackers break SSL encryption used by millions of sites

Beware of BEAST decrypting secret PayPal cookies

By **Dan Goodin in San Francisco** • **Get more from this author**

Posted in Security, 19th September 2011 21:10 GMT

Home > Security

News

'CRIME' attack abuses SSL/TLS data compression feature to hijack HTTPS sessions

SSL/TLS data compression leaks information that can be used to decrypt HTTPS session cookies, researchers say

By Lucian Constantin

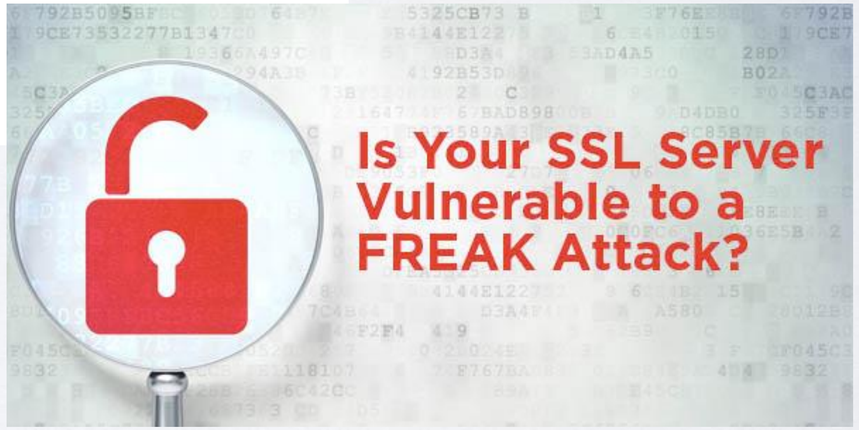
September 13, 2012 09:02 PM ET Add a comment

A brief history

- **TLSv1.1, TLS v1.2 (2006, 2008)**
  - Fixed some of this, but…
  - Downgrade attacks!

This POODLE Bites: Exploiting The SSL 3.0 Fallback

Security Advisory



TLS flaws (in a nutshell)

- **Bad protocol design**
- **Legacy ciphers and cipher suites**
- **Poor parameter selection**
- **Weak implementations**

What's wrong with TLS?

Quite a bit

- Many problems result from TLS's use of "***pre-historic cryptography***" (- Eric Rescorla)
  - CBC with Mac-then-Encrypt, bad use of IVs
  - RSA-PKCS#1v1.5 encryption padding
  - RC4
  - DH parameter generation
  - Horrifying backwards compatibility requirements

Quite a bit

- Many problems result from TLS's use of "***pre-historic cryptography***" (- Eric Rescorla)
  - CBC with Mac-then-Encrypt, bad use of IVs
  - RSA-PKCS#1v1.5 encryption padding
  - RC4
  - D
  - H

Many of these flaws were 'known' at design time, but exploited by researchers only afterwards.

MAC-then-pad-then-Encrypt

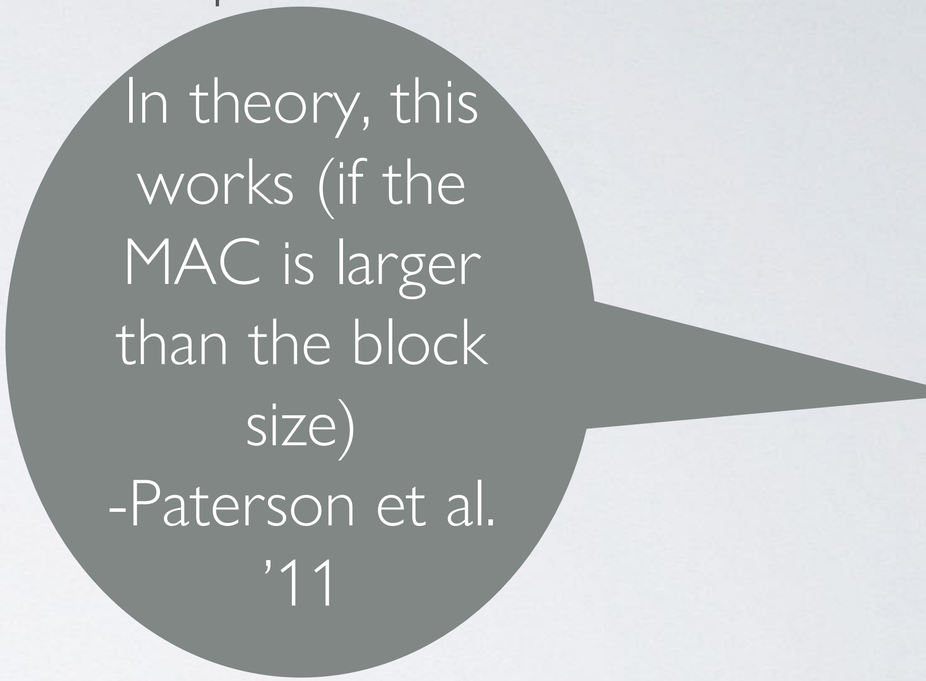
- TLS MACs the record, then pads (in CBC), then enciphers
  - Obvious problem: padding oracles (2002)

MAC-then-pad-then-Encrypt

- TLS MACs the record, then pads (in CBC), then enciphers
  - Obvious problem: padding oracles
  - Countermeasure(s):
    1. Do not distinguish padding/MAC failure

MAC-then-pad-then-Encrypt

- TLS MACs the record, then pads (in CBC), then enciphers
  - Obvious problem: padding oracles
  - Countermeasure(s):
    1. Do not distinguish padding/MAC failure



MAC-then-pad-then-Encrypt

- TLS MACs the record, then pads (in CBC), then enciphers
  - Obvious problem: padding oracles
  - Countermeasure(s):
    1. Do not distinguish padding/MAC failure
    2. “Constant-time” decryption

MAC-then-pad-then-Encrypt

- TLS MACs the record, then pads (in CBC), then enciphers
  - Obvious problem: padding oracles
  - Countermeasure(s):
    1. Do not distinguish padding/MAC failure
    2. “Constant-time” decryption

Unlucky for you: UK crypto-duo 'crack' HTTPS in Lucky 13 attack

OpenSSL patch to protect against TLS decryption boffinry

By **John Leyden** • **Get more from this author**

Posted in Security, 4th February 2013 16:58 GMT

BEAST

- Serious bug in TLS 1.0
- Allows complete decryption of CBC ciphertexts
- Use of predictable Initialization Vector (CBC residue bug)
  - Known since 2002, attack described by Bard in 2005 (***Bard was advised to focus on more interesting problems***.)
  - Nobody cared or noticed until someone implemented it

The RC4 workaround

After BEAST
Some deployments favored RC4 to avoid TLS 1.0 CBC weaknesses.

A bad trade
RC4 has its own serious statistical weaknesses.

Since 2015
RFC 7465 prohibits RC4 cipher suites in TLS.

Modern design
TLS 1.3 uses authenticated encryption (AEAD).

Compression and encryption

CRIME (2012)
Compressed lengths can reveal secrets when an attacker controls related input.

The TLS response
TLS 1.3 removes TLS record compression.

The broader lesson
Application-level compression can still leak through encrypted lengths.

Security depends on context
Ask what the attacker controls and what lengths they can observe.

Protocol Design

Example: Negotiation

In TLS 1.2 and earlier, cipher-suite negotiation selects the key exchange and record-protection algorithms.



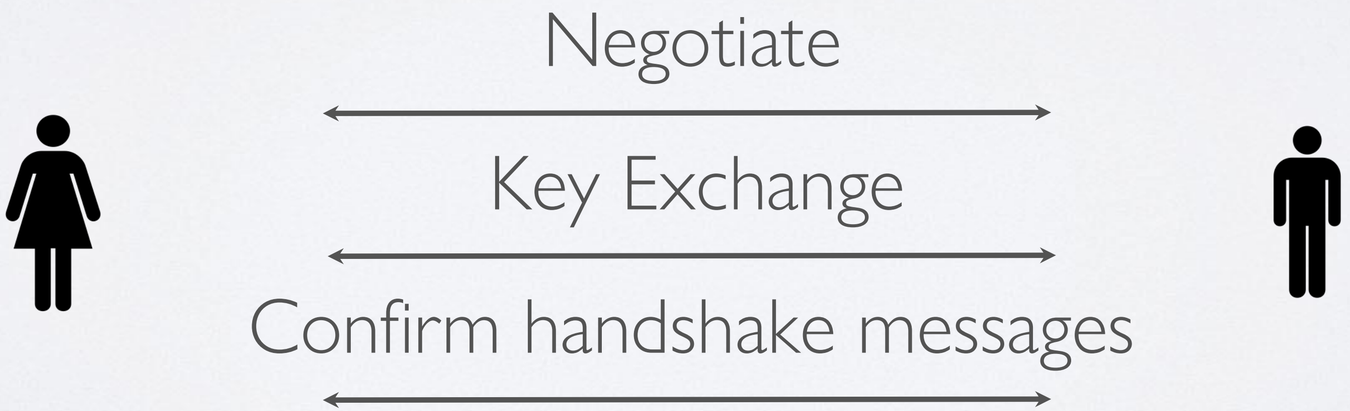
Example: Negotiation

In TLS 1.2 and earlier, cipher-suite negotiation selects the key exchange and record-protection algorithms.

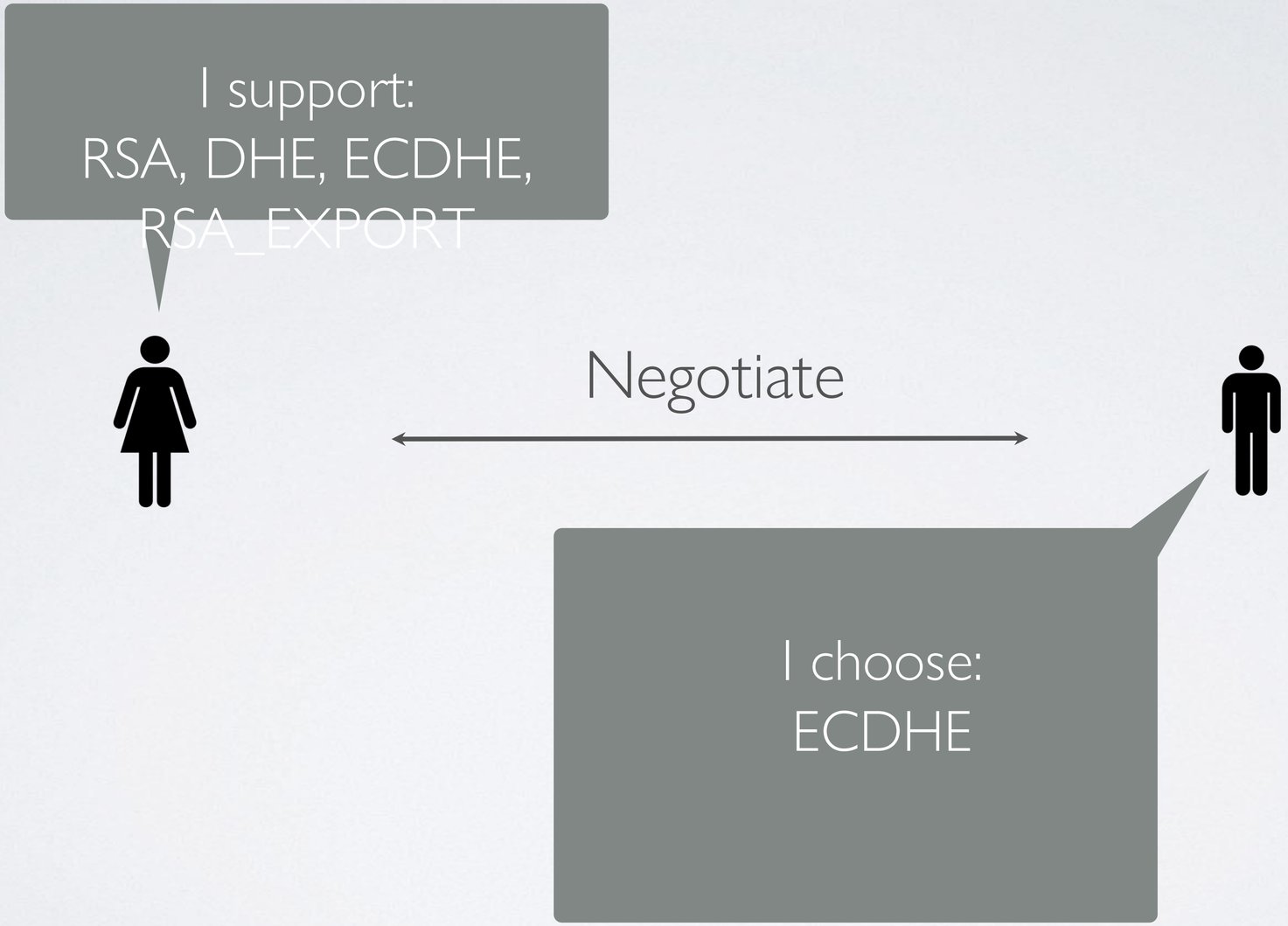


Example: Negotiation

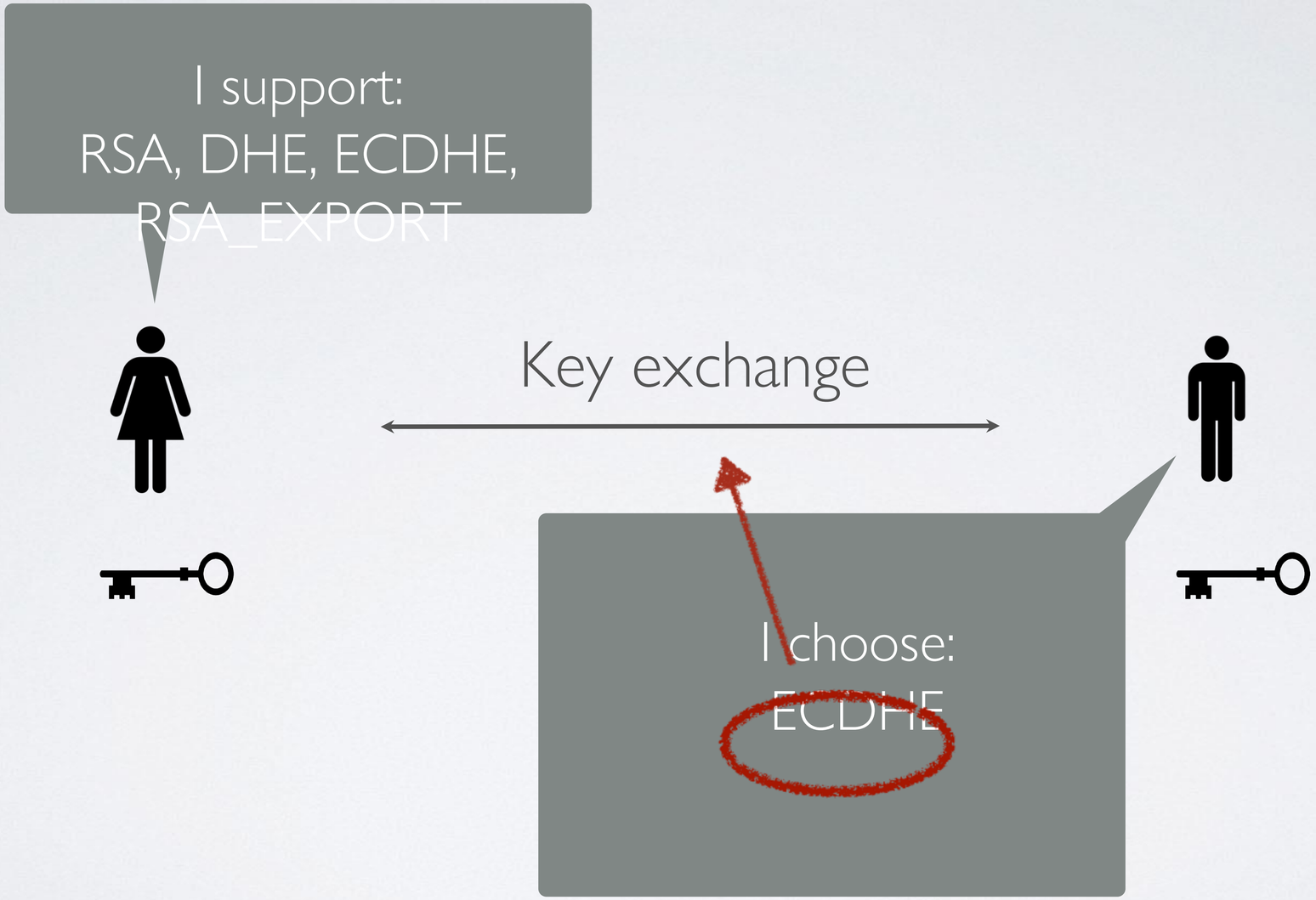
In TLS 1.2 and earlier, cipher-suite negotiation selects the key exchange and record-protection algorithms.



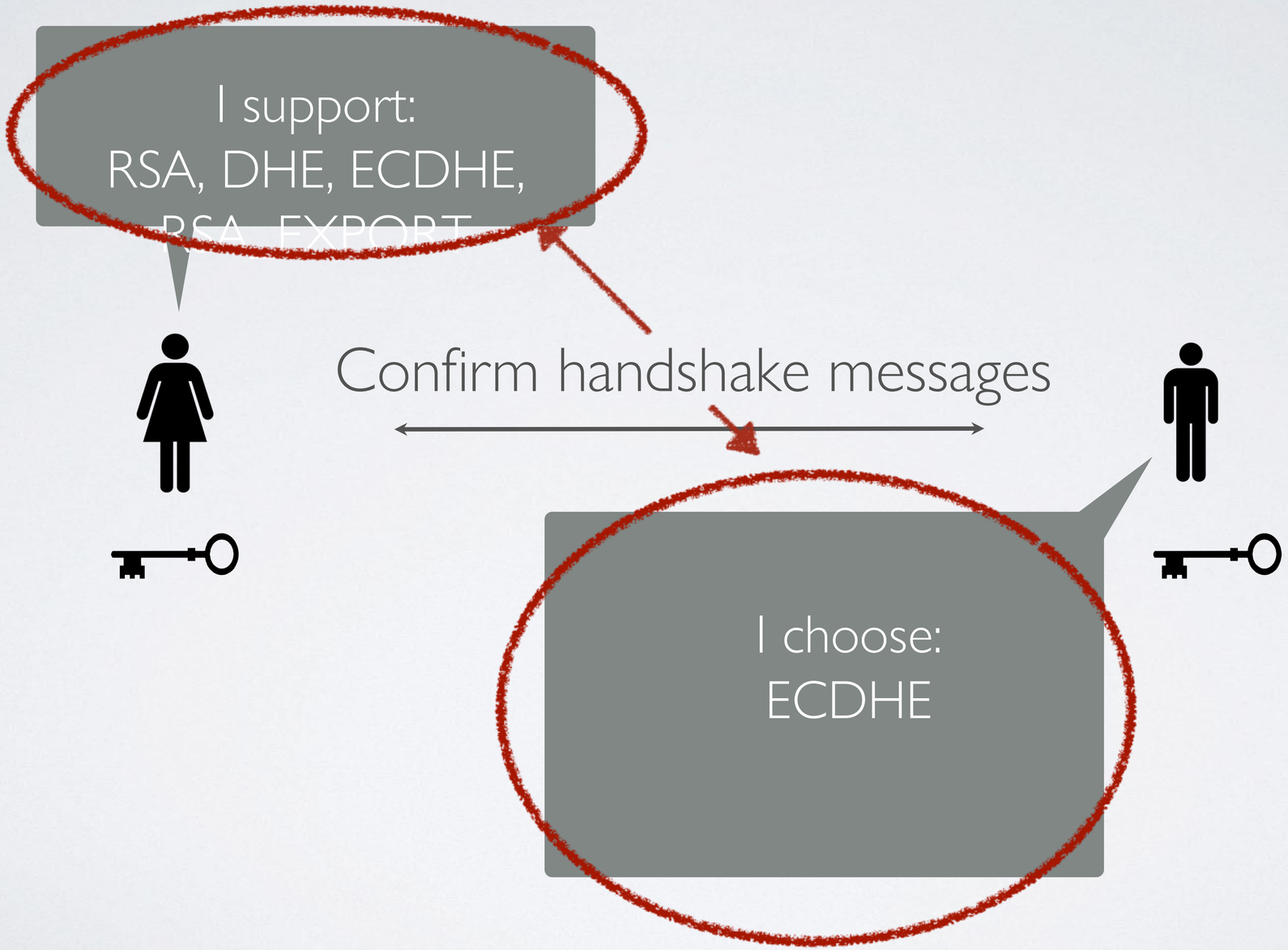
Ciphersuite Negotiation



Ciphersuite Negotiation



Ciphersuite Negotiation



MITM Negotiation



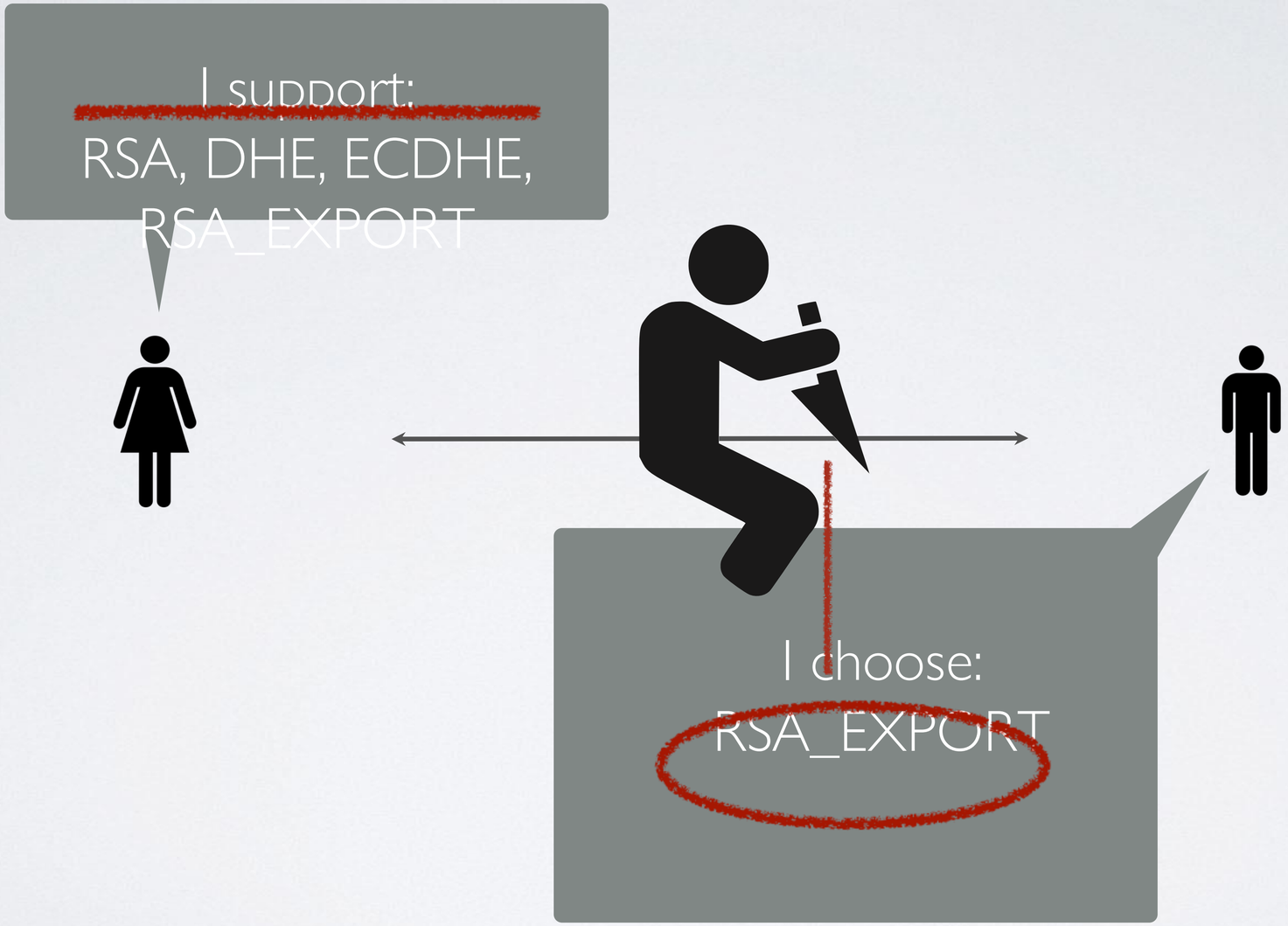
MITM Negotiation



MITM Negotiation



MITM Negotiation



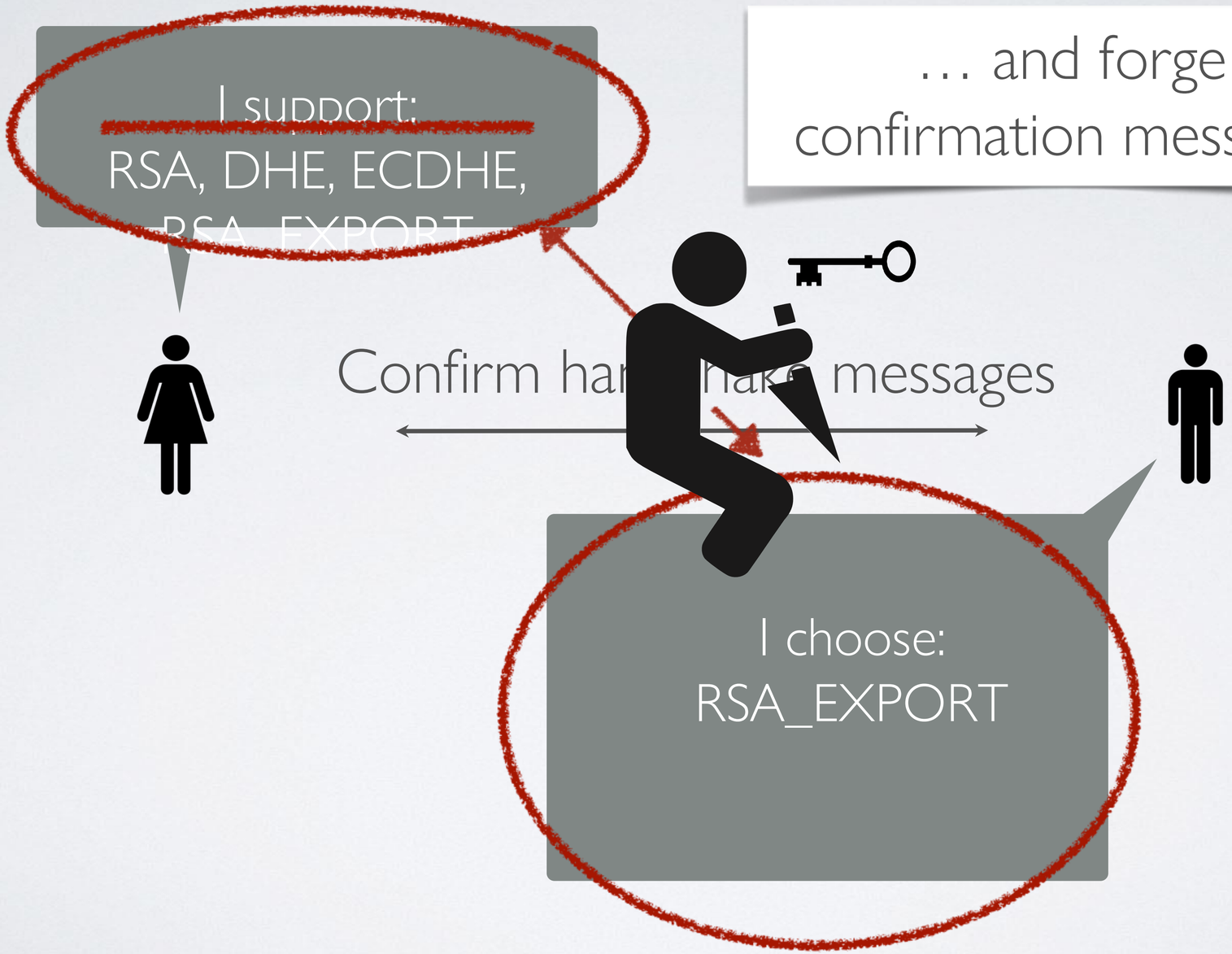
MITM Negotiation

I support:
RSA, DHE, ECDHE, RSA_EXPORT

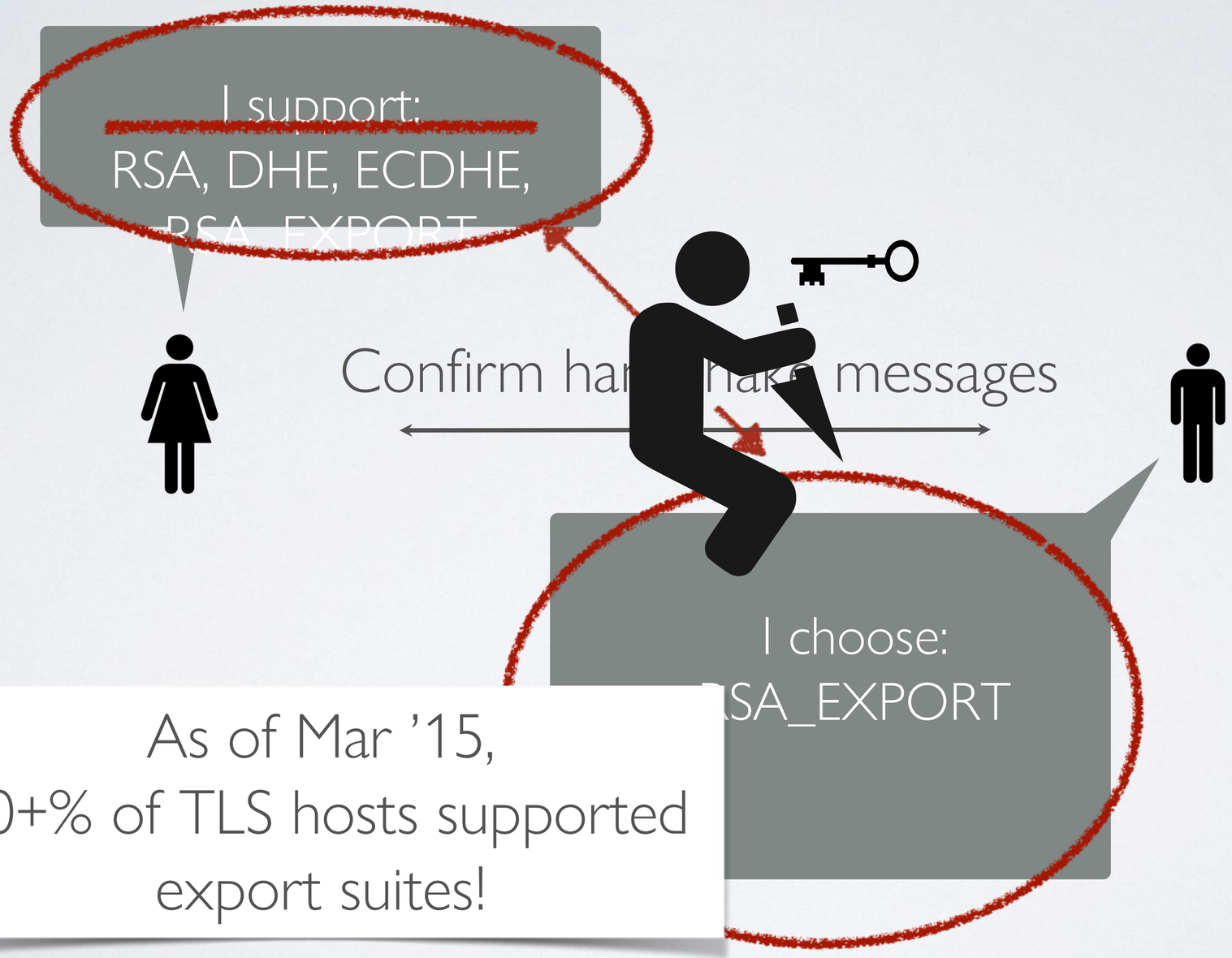
Attacker can break RSA export key



MITM Negotiation



MITM Negotiation



As of Mar ’15,
30+% of TLS hosts supported
export suites!

MITM Negotiation

I support:

Solution:

Disable export suites and reject unoffered key exchanges.
Both peers must enforce the policy.

I choose:
RSA_EXPORT

As of Mar ’15,
30+% of TLS hosts supported export suites!

Question

Is it sufficient for the client to support only “strong” ciphersuites, even if the server supports weak ones?

Question

Is it sufficient for the client to support only "strong" ciphersuites, even if the server supports weak ones?

- Let ***A*** be the set of KA protocols supported by Client
  Let ***B*** be the set of KA protocols supported by Server

- If each KA protocol in $A \cap B$ is a secure KA protocol, is the TLS handshake secure

TLS for cryptographers

- In CRYPTO 2012 (!) we saw the first paper to successfully analyze TLS-DHE [Jager ***et al.]***
- In CRYPTO 2013 a random-oracle analysis of the TLS-RSA handshake [Krawczyk ***et al.]***
- In CRYPTO 2014 an automated analysis of the full handshake, under a new security model [Bhargavan ***et al.]***

TLS for cryptographers

We do not model ciphersuite negotiation/renegotiation, nor session resumption.

- In CRYPTO 2012 (!) we saw the first paper to successfully analyze TLS-DHE [Jager ***et al.]***
- In CRYPTO 2013 a random-oracle analysis of the TLS-RSA handshake [Krawczyk ***et al.]***
- In CRYPTO 2014 an automated analysis of the full handshake, under a new security model [Bhargavan ***et al.]***

TLS for cryptographers

- In CRYPTO 2012 (!) we saw the first paper to successfully analyze TLS-DHE [Jager ***et al.]***
- In CRYPTO 2013 a random-oracle analysis of the TLS-RSA handshake [Krawczyk ***et al.]***
- In CRYPTO 2014 an automated analysis of the full handshake, under a new security model [Bhargavan ***et al.]***

Theorem

- **Bhargavan *et al.* theorem statement:**
  Let ***A*** be the set of KA protocols supported by Client
  Let ***B*** be the set of KA protocols supported by Server

  If each KA protocol in $A \cup B$ is a secure KA protocol & there exist PRFs, then the TLS handshake is a secure KA protocol.

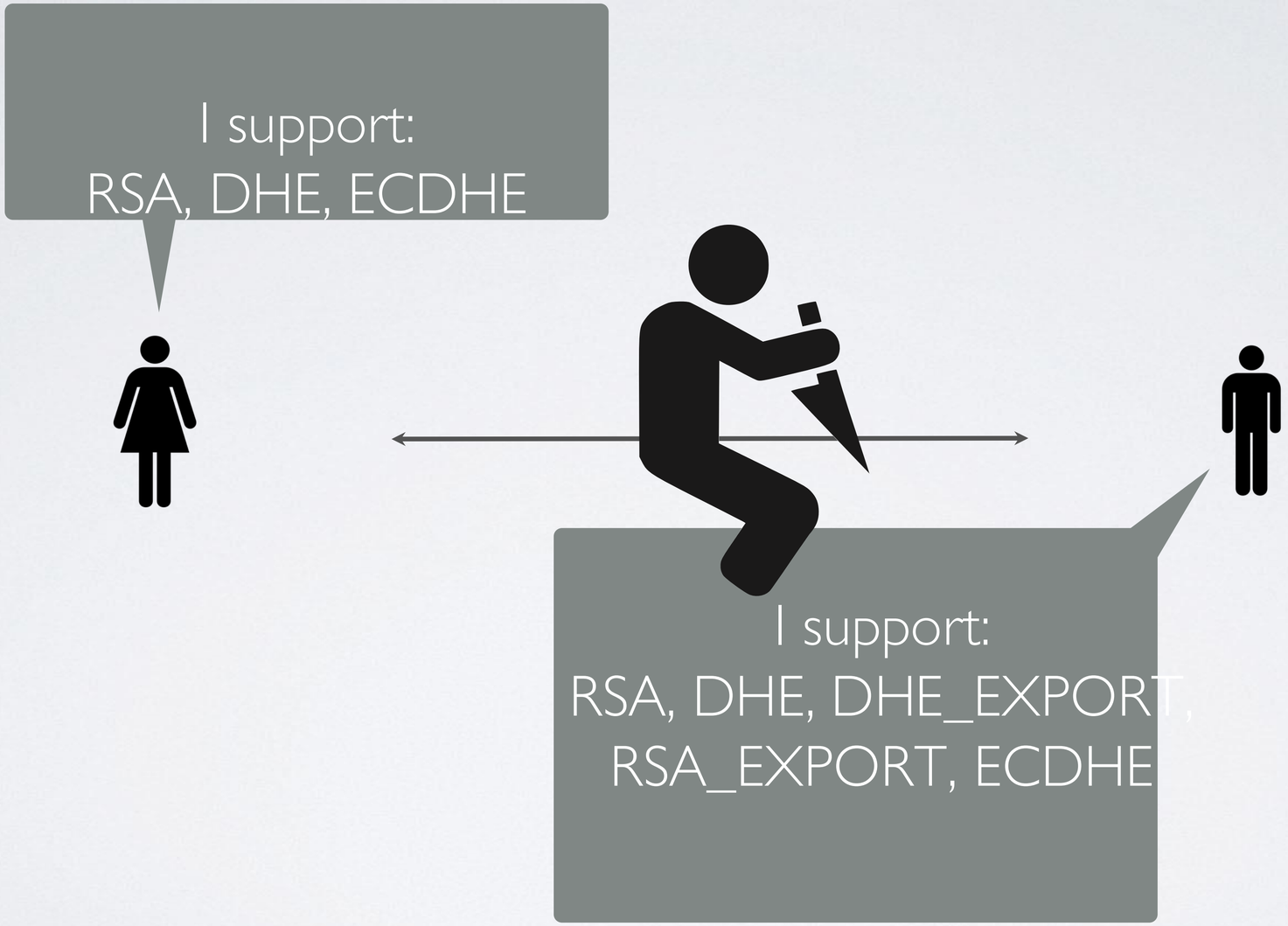
Theorem

- **Bhargavan *et al.* theorem statement:**
  Let ***A*** be the set of KA protocols supported by Client
  Let ***B*** be the set of KA protocols supported by Server

  If each KA protocol in $A \cup B$ is a secure KA protocol & there exist PRFs, then the TLS handshake is a secure KA protocol.

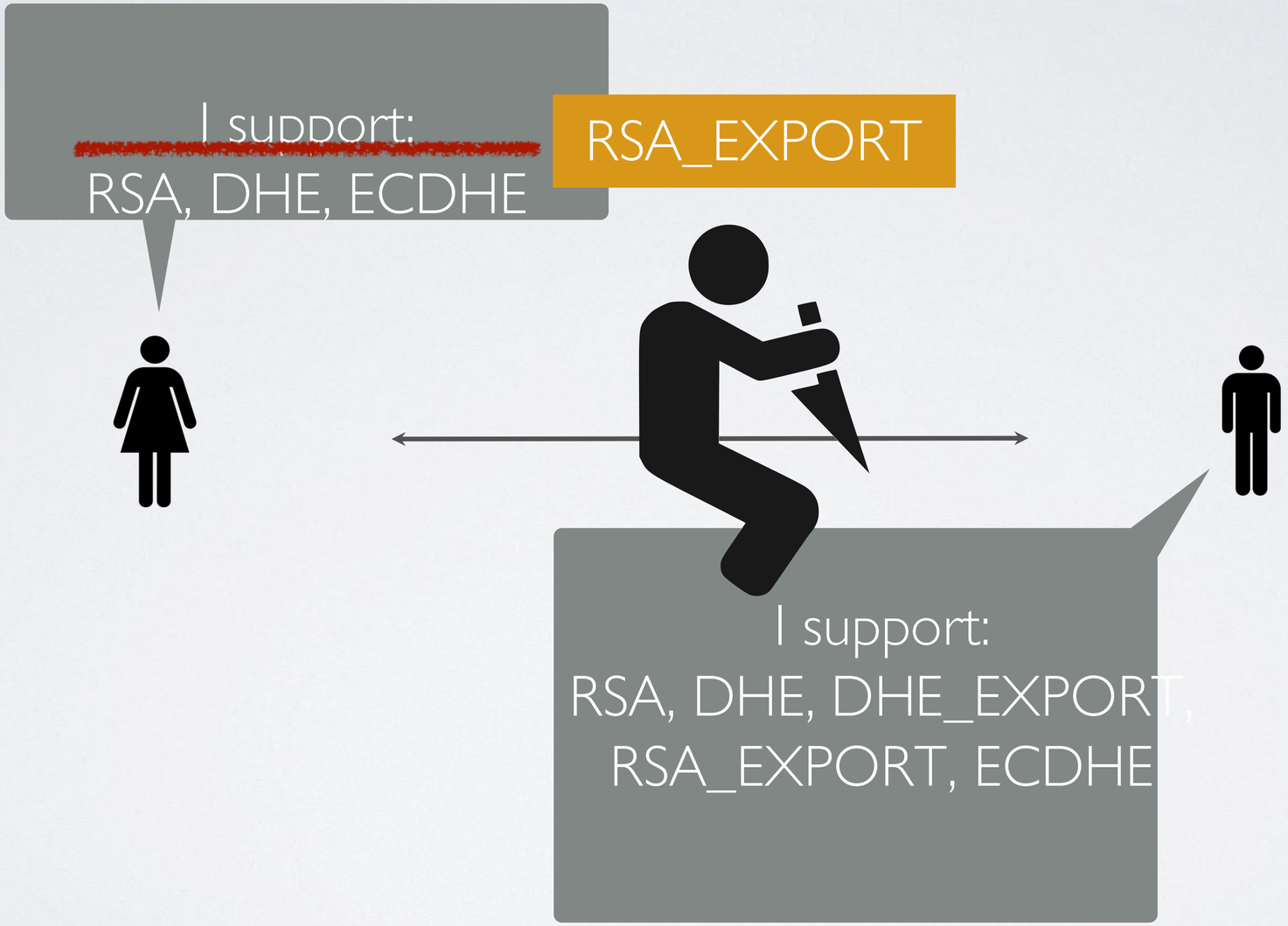
TLS design/deployment assumes this would be $A \cap B$!

Example 2: Negotiation



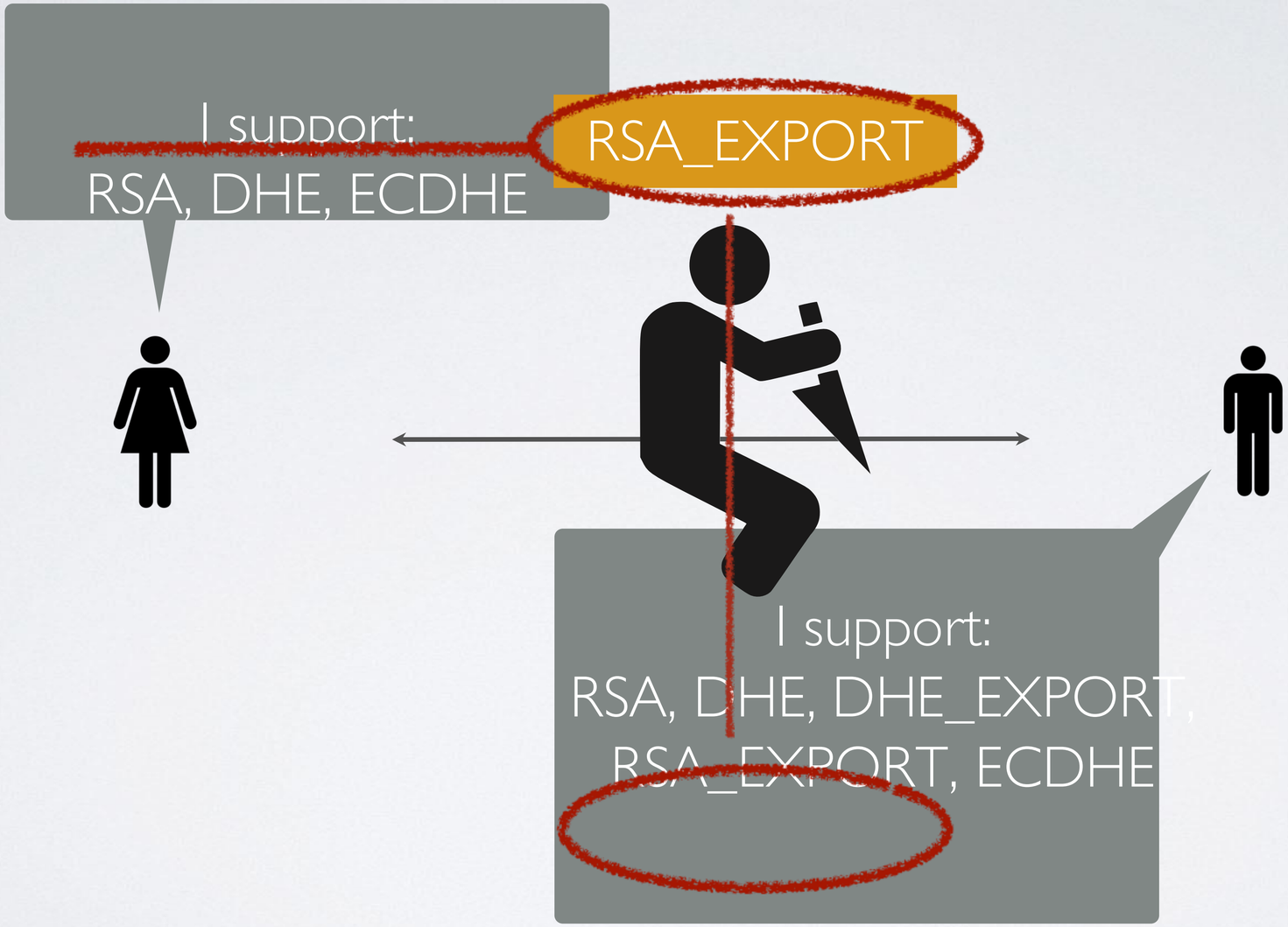
Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 2: Negotiation



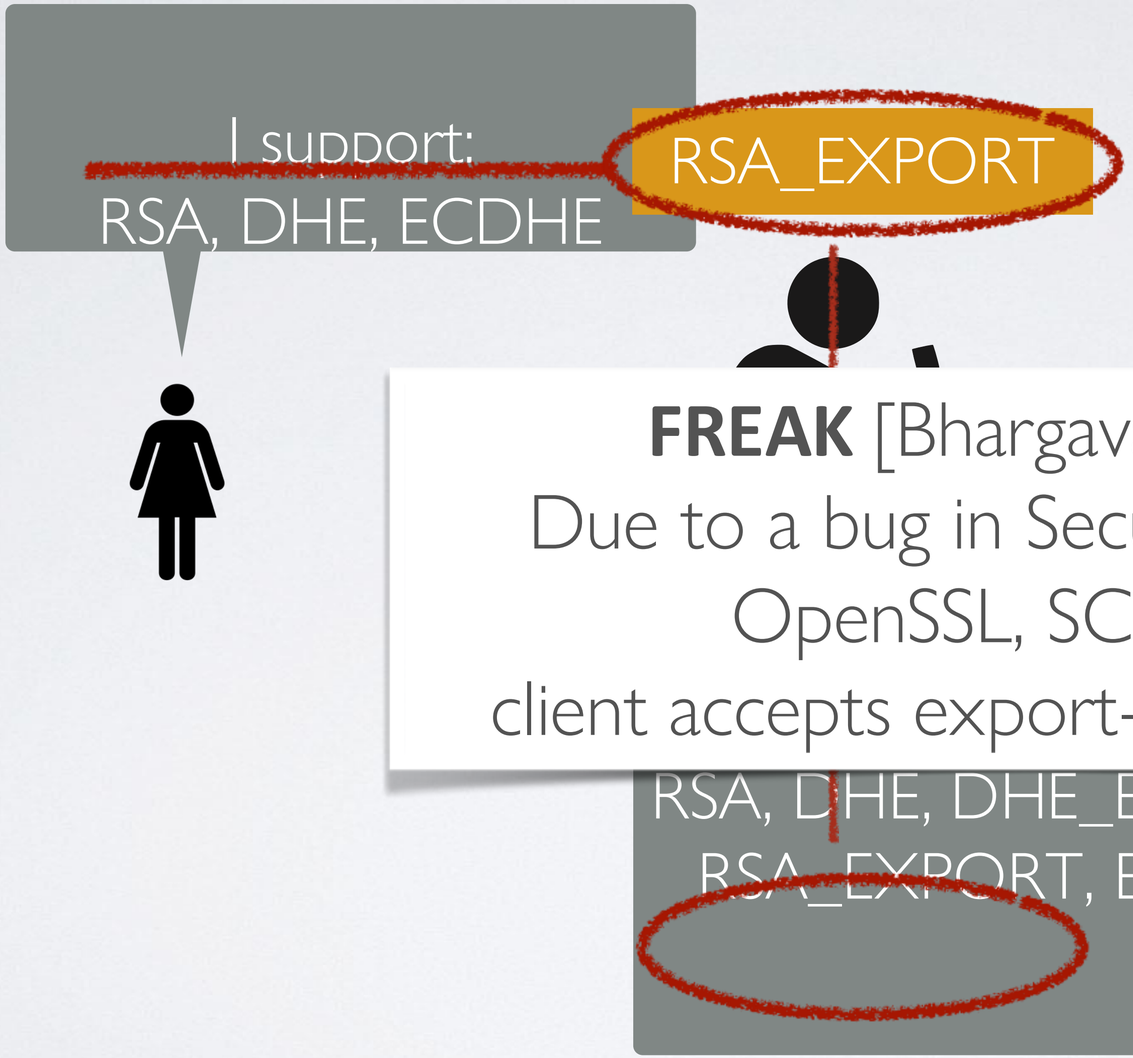
Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 2: Negotiation



Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 2: Negotiation



FREAK [Bhargavan *et al.]*:
Due to a bug in SecureTransport,
OpenSSL, SChannel,
client accepts export-grade RSA key

Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Historical case study: TLS 1.2 and earlier (1996–2015)



Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 2: Negotiation

Solution: Fix implementations

Patch OpenSSL, SecureTransport, SChannel so they will recognize an RSA export key exchange message, barf

(patches rolled out March 2015)

RSA_DHE_DHE_EXPORT

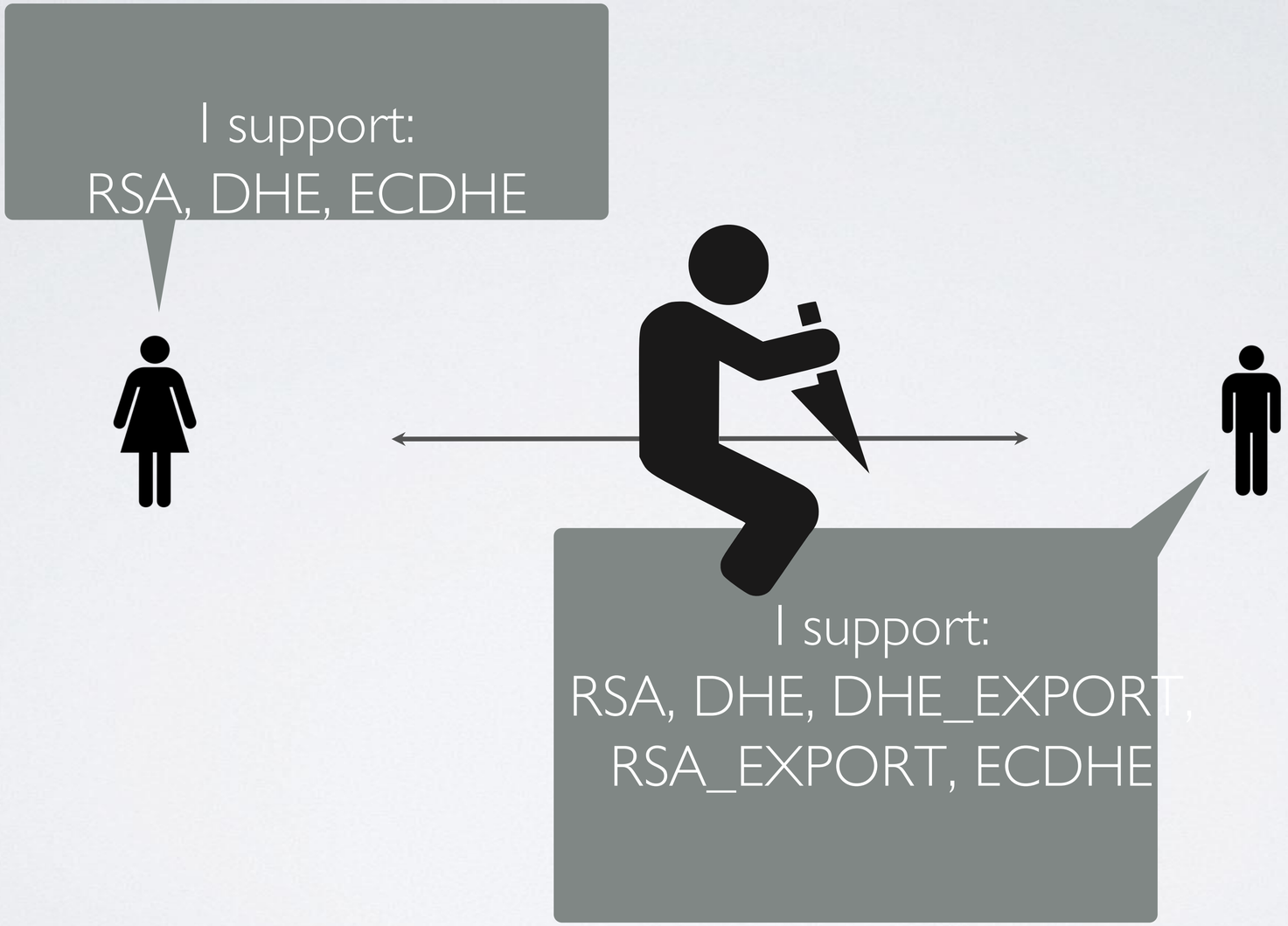
Apple issues security patches to protect devices from the FREAK bug



by Mariella Moon | @mariella_moon | 22 days ago

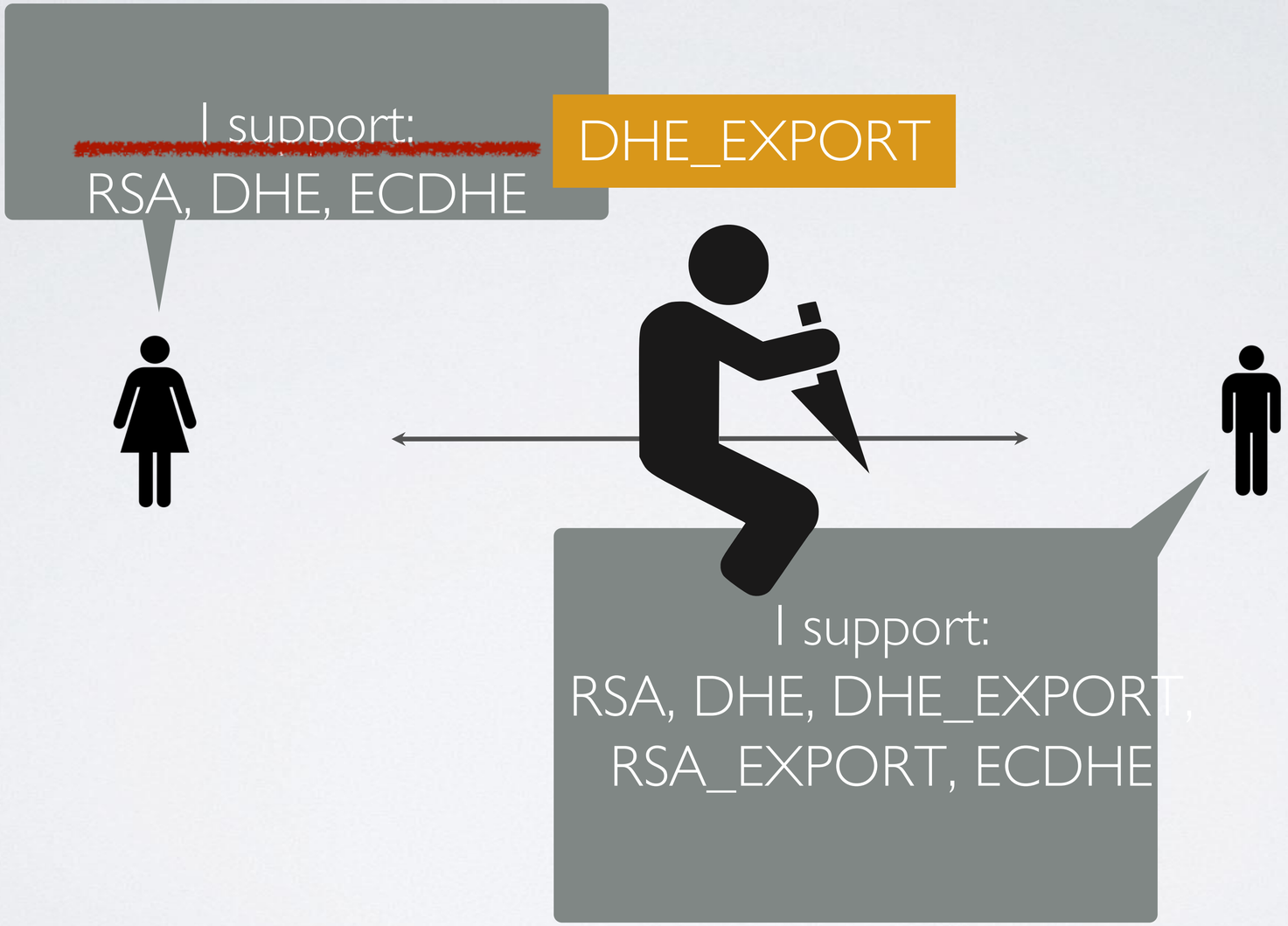
Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 3: Negotiation



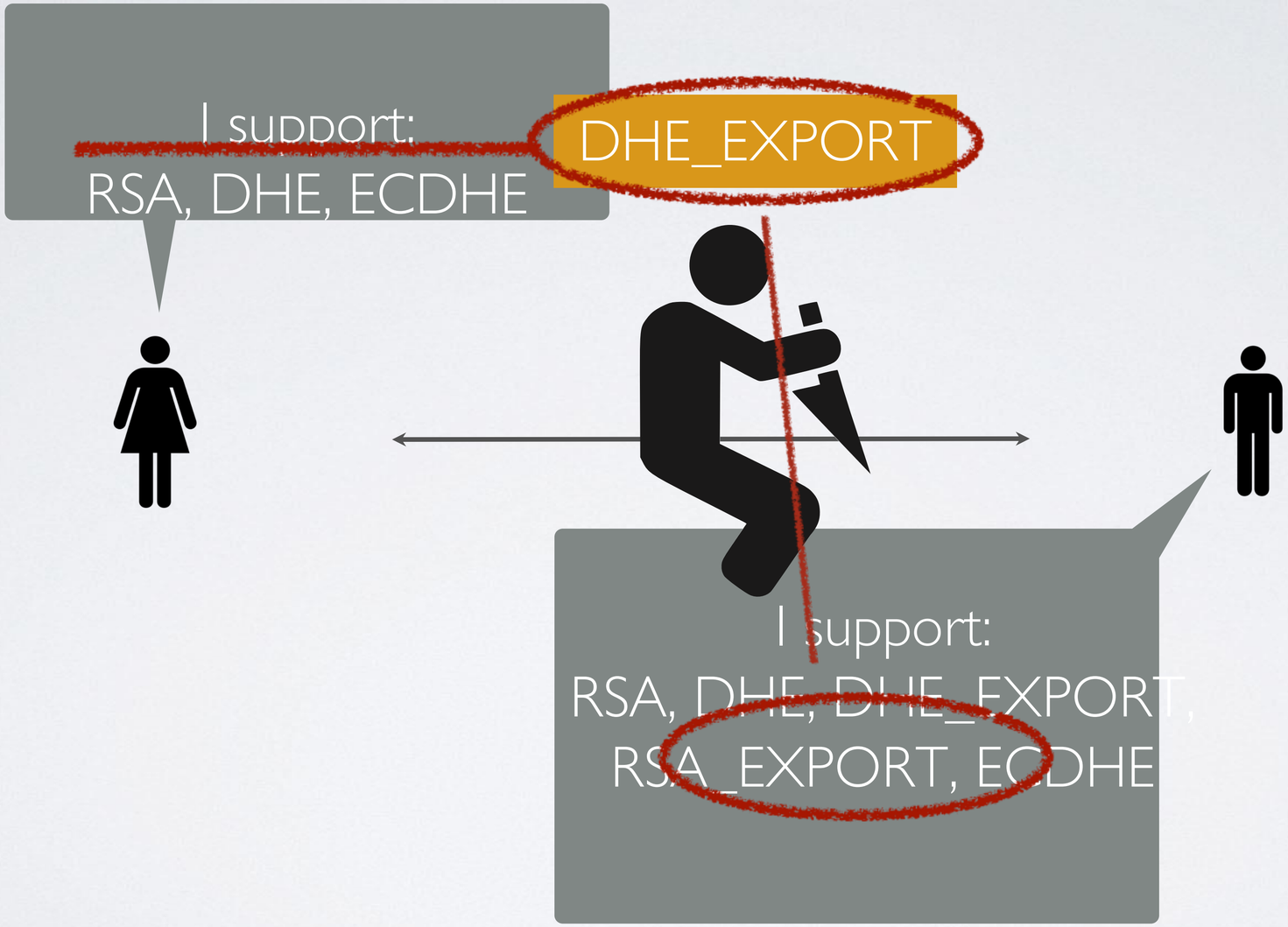
Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 3: Negotiation



Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 3: Negotiation



Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 3: Negotiation

```
struct {
    select (KeyExchangeAlgorithm) {
        case dh_anon:
            ServerDHParams params;
        case dhe_dss:
        case dhe_rsa:
            ServerDHParams params;
            digitally-signed struct {
                opaque client_random[32];
                opaque server_random[32];
                ServerDHParams params;
            } signed_params;
        case rsa:
        case dh_dss:
        case dh_rsa:
            struct {} ;
           /* message is omitted for rsa, dh_dss, and dh_rsa */
        /* may be extended, e.g., for ECDH -- see [TLSECC] */
    };
} ServerKeyExchange;
```

Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

Example 3: Negotiation



LogJam [Adrian *et al.]*:
Due to a bug in ***the TLS protocol***
client accepts export-grade DHE key

Tolga Acar, Mira Belenkiy, Mihir Bellare, and David Cash, Cryptographic Agility and its Relation to Circular Encryption, in EUROCRYPT 2010

TLS design/deployment assumptions were wrong, and we knew this for years — but failed to properly communicate to the community.

TLS design/deployment assumptions were wrong, and we knew this for years — but failed to properly communicate to the community.

The community made terrible assumptions and didn't ask us what we thought of them. Then they got mired in backwards compatibility issues and only responded to attacks.

Exploiting LogJam

Exploiting LogJam

- To exploit the downgrade attack, requires solving a 512-bit DL in real time
  - Initially this seems challenging, but NFS algorithm can be heavily optimized for pre-computation using only prime (p)
  - "Oversieving" increases cost of sieving and storage, but reduces cost of linear algebra step & final "descent"

Exploiting LogJam

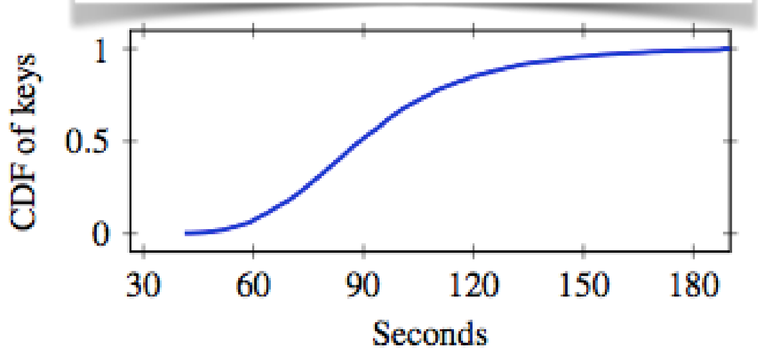
- To exploit the downgrade attack, requires solving a 512-bit DL in real time
  - **92% of DHE_EXPORT servers use one of two hard-coded primes (*p) (Mod_SSL, Apache)***

Exploiting LogJam

- To exploit the downgrade attack, requires solving a 512-bit DL in real time
  - **92% of DHE_EXPORT servers use one of two hard-coded primes (*p*) *(Mod_SSL, Apache)***

Sieving/Linear Alg:
1 week (wall clock) for each ***p***

Descent on (g, h)



Logjam: the immediate response

The 2015 downgrade
An attacker forced export DHE and solved a 512-bit discrete logarithm.

Emergency patches
Clients rejected very small DH groups, including the 768-bit cutoff discussed here.

Compatibility cost
Raising minimum sizes broke some legacy deployments.

Historical threshold
768 bits was an emergency barrier, not a secure modern target.

Logjam: lasting lessons

Parameter strength
Small DH groups made precomputation economically attractive.

Negotiation must be bound
Authenticate the negotiated choices and the key exchange together.

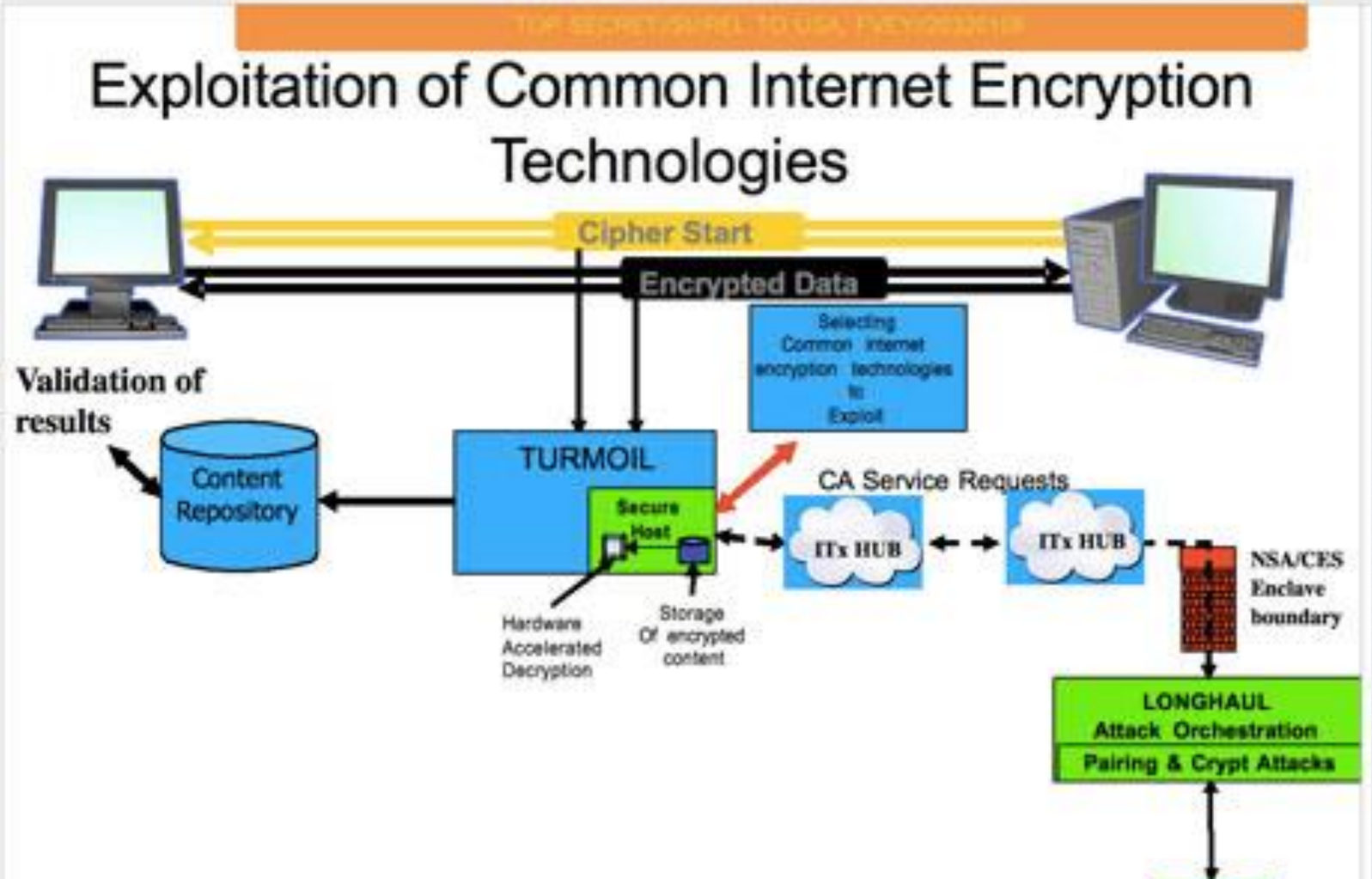
TLS 1.3
CertificateVerify signs a transcript hash. Finished confirms the shared keys.

The missing step
The client must also validate the certificate and intended server identity.

Historical hypothesis: LONGHAUL (2015), mechanism not established

This picture again

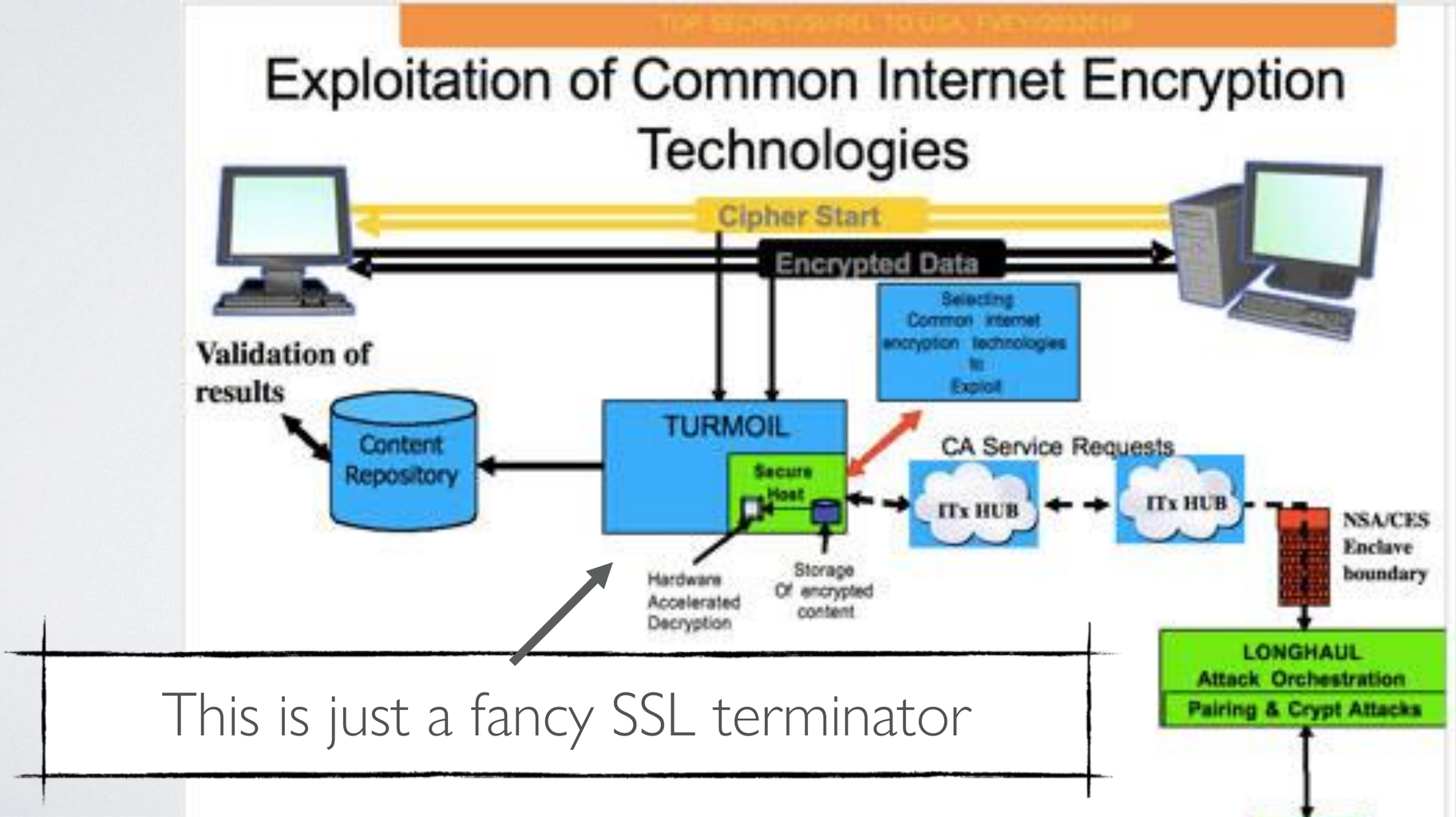
- **What's going on here?**



Historical hypothesis: LONGHAUL (2015), mechanism not established

This picture again

- **What's going on here?**

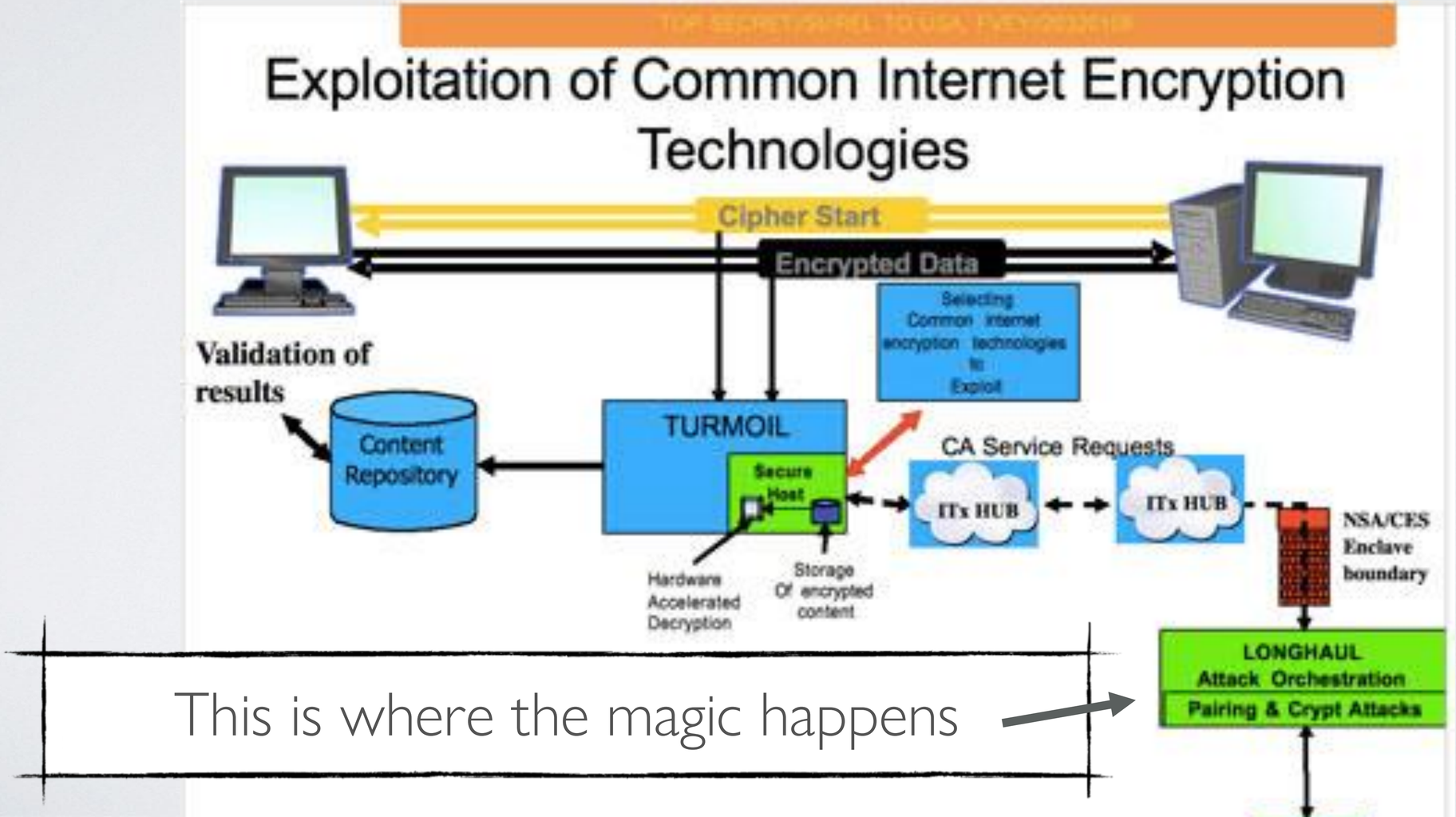


This is just a fancy SSL terminator

Historical hypothesis: LONGHAUL (2015), mechanism not established

This picture again

- **What's going on here?**



This is where the magic happens

Historical hypothesis: LONGHAUL (2015), mechanism not established

What is LONGHAUL?

(TS//SI/REL) The **LONGHAUL** system provides the **Extended NSA Enterprise** with an end-to-end attack orchestration and key recovery service for Data Network Cipher (DNC) and Data Network Session Cipher (DNSC) traffic. LONGHAUL is extensible to allow for the addition of other Digital Network Intelligence cipher types.

Hypothesis 1: LONGHAUL is a database of stolen RSA secret keys

- This works well, but it's boring
- Easy to solve: switch to PFS cipher suites (DHE/ECDHE)

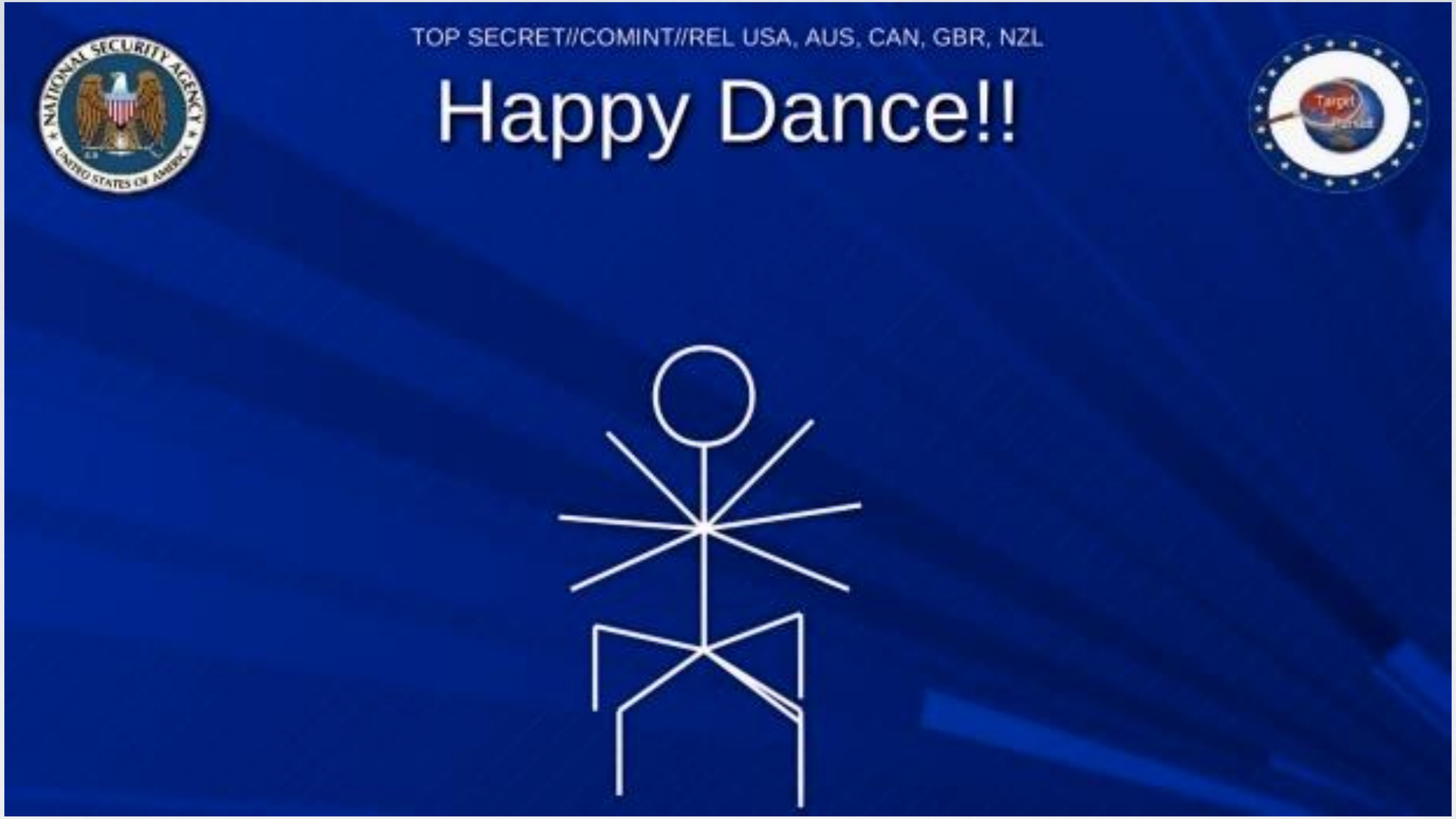
Historical hypothesis: LONGHAUL (2015), mechanism not established

RSA Exploitation Steps

- Is it the key exchange RSA? (server hello)
  - If so, is the modulus match a known private key? (server certificate)
    - If so, is there 2-sided collect?
      - If so, do we have:
        - Client Hello
        - Server Hello
        - Client Key Exchange

DECRYPTION!

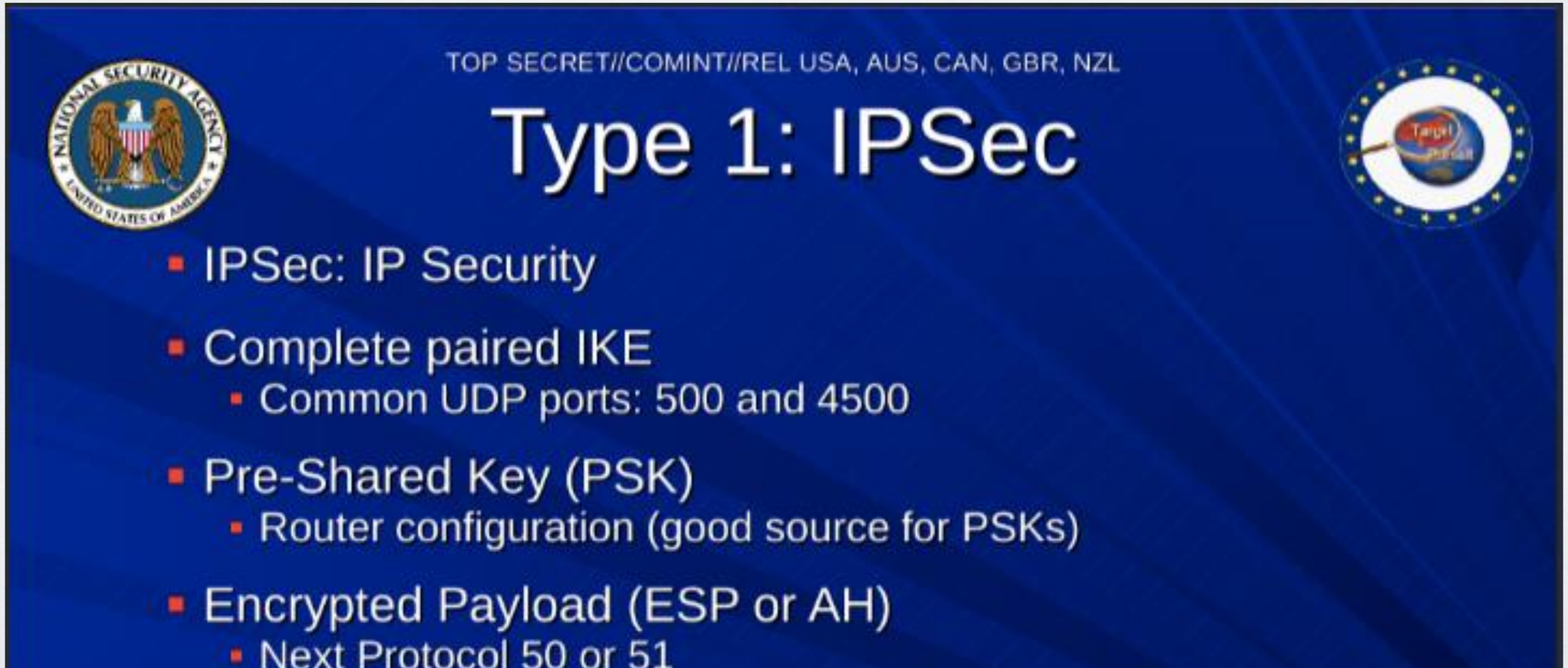
Historical hypothesis: LONGHAUL (2015), mechanism not established



Historical hypothesis: LONGHAUL (2015), mechanism not established

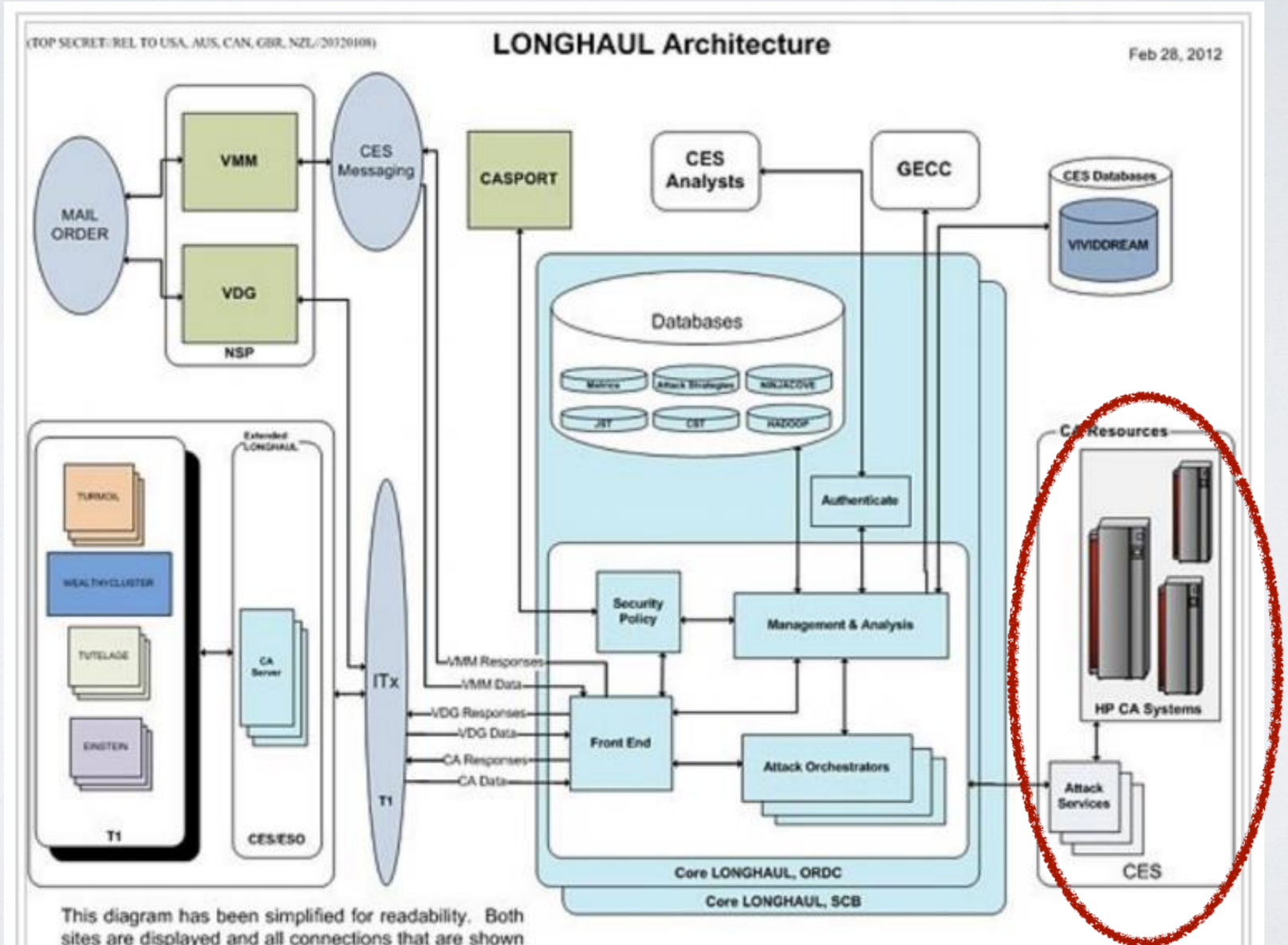
Problem

- LONGHAUL also purports to decrypt IPSec/IKE
  - IKE uses DH for key agreement.
  - RSA signatures may authenticate the exchange.



Historical hypothesis: LONGHAUL (2015), mechanism not established

What is LONGHAUL?



Historical hypothesis: LONGHAUL (2015), mechanism not established

The breakthrough was enormous, says the former official, and soon afterward the agency pulled the shade down tight on the project, even within the intelligence community and Congress. “Only the chairman and vice chairman and the two staff directors of each intelligence committee were told about it,” he says. The reason? “They were thinking that this computing breakthrough was going to give them the ability to crack current public encryption.”

Historical hypothesis: LONGHAUL (2015), mechanism not established

What is LONGHAUL?

(TS//SI/REL) The **LONGHAUL** system provides the **Extended NSA Enterprise** with an end-to-end attack orchestration and key recovery service for Data Network Cipher (DNC) and Data Network Session Cipher (DNSC) traffic. LONGHAUL is extensible to allow for the addition of other Digital Network Intelligence cipher types.

Hypothesis 2: The NSA is breaking1024-bit DHE

- This sounds completely insane
- Maybe it's not

DH precomputation at scale

The reusable work
Number-field-sieve precomputation targets a particular prime.

The per-connection work
A final descent solves the discrete logarithm for an individual public value.

768 bits is not a safety boundary
Public computations reached 768 bits in 2016 and 795 bits in 2019.

What follows
Avoid undersized groups. Reusing a sufficiently strong standard group is acceptable.

Breaking DHE at scale

- A large percentage of Apache/Java/ISS servers use ***fixed, hardcoded parameters for DHE***
  - IPSec/IKE is even worse: nearly 50% of servers will choose Oakley groups 1 and 2 (768/1024) - generated in 1998
  - NFS is heavily optimized for pre-computation using only the primes
  - **With specific pre-computation ($10s-100s of Million/1 year?) an attacker might be able to break 30-50% of DHE connections <u>with academic levels of computing</u>**
  - Approximately 30 core days for final descent



DH guidance today

Prefer modern TLS
Use TLS 1.3. For TLS 1.2, prefer ECDHE with AEAD.

Finite-field DH
Use at least 2048 bits where finite-field DH is required.

Standard groups
RFC 7919 defines shared, vetted groups. Unique primes are not the general remedy.

Forward secrecy
RSA key transport is not a modern fallback for weak DH.

Next: how TLS 1.3 implements these lessons.

TLS 1.3: what changed

Record protection
AEAD replaces the old CBC/MAC and RC4 constructions.

Key establishment
Fresh public-key exchanges provide forward secrecy. RSA key transport is gone.

Negotiation
Cipher suites select AEAD + hash. Groups and signatures are chosen separately.

Less legacy machinery
TLS compression and renegotiation are removed.

TLS 1.3: RFC 8446 (2018), updated by RFC 9846 (2026).

A TLS 1.3 handshake

1. Client to server
ClientHello: supported choices + an ephemeral key share.

2. Server to client
ServerHello: selected choices + server key share.
Then encrypted extensions, certificate, CertificateVerify, Finished.

3. Client to server
Validate the server, then send Finished and application data.

Simplified full handshake: server authentication, no retry or client certificate.

What authenticates the exchange?

Certificate validation
Does this trusted certificate identify the server I intended to contact?

CertificateVerify
Does that server possess the signing key and endorse this transcript?

Finished
Did the peer derive the expected keys from this exchange and transcript?

Tampering changes the transcript
Changing offered groups, selected parameters, or key shares breaks verification.

Encryption by itself does not establish who is at the other end.

Records, resumption, and 0-RTT

Traffic keys
HKDF derives separate client/server keys and separates handshake from application traffic.

Records
AEAD protects confidentiality and integrity. Sequence numbers help form nonces.

Resumption
A ticket allows a later connection to use a resumption secret.

0-RTT early data
An attacker can replay it. Applications must decide which requests are safe to repeat.

QUIC also uses TLS 1.3, with its own packet protection.

Hybrid ML-KEM in TLS 1.3

ClientHello key share
Fresh ML-KEM encapsulation key + ephemeral X25519 public value.

ServerHello key share
ML-KEM ciphertext + ephemeral X25519 public value.

Both sides compute
The same ML-KEM secret and the same X25519 secret.

TLS key schedule
HKDF combines the secrets with the TLS handshake context to derive traffic keys.

Example group: X25519MLKEM768 (RFC 10024, August 2026).

What the hybrid buys us

A future decryption threat
An attacker may record traffic today and later use a quantum computer to break ECDHE.

Two sources of secrecy
The hybrid aims to preserve key secrecy if either component remains secure.

Authentication is a separate job
Certificates and handshake signatures still authenticate the server. ML-KEM is not a signature.

Same application encryption
AES-GCM or ChaCha20-Poly1305 still protects the data. ML-KEM establishes keys.

Post-quantum key establishment does not make classical signatures post-quantum.

Surely this is all the IETF's fault

Case study: Apple iMessage

- **Not the most important security protocols on the Internet**
  - But pretty important to real people
  - Once you have messaging, you can build inter-device communications…

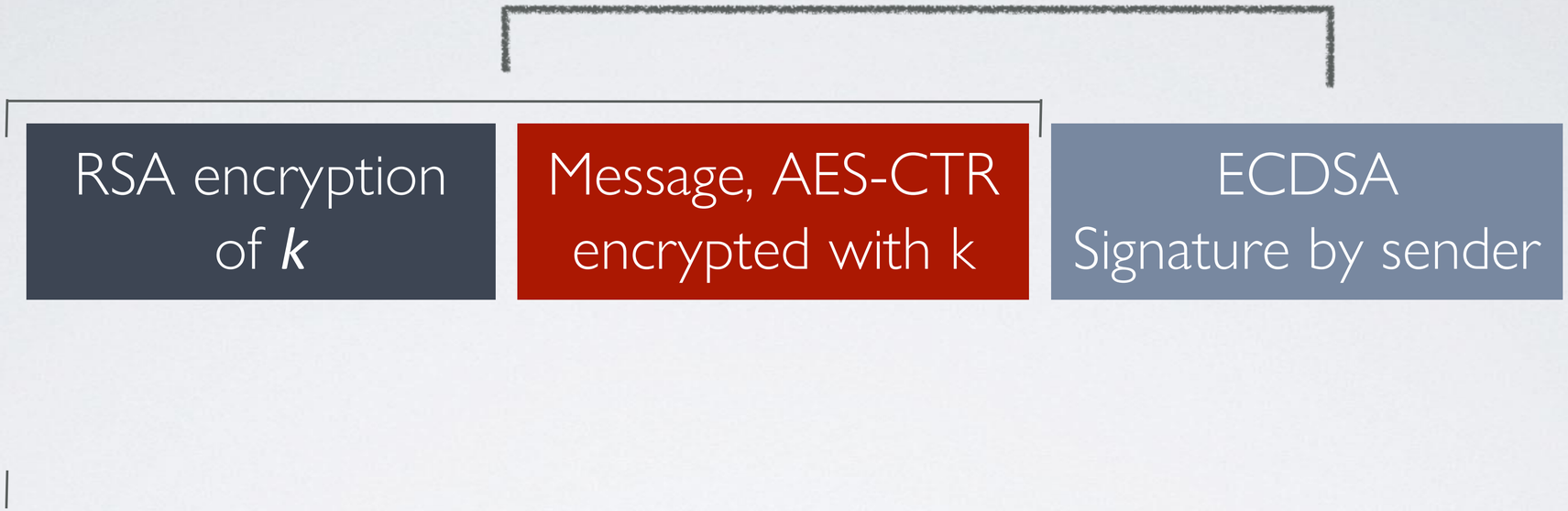


iMessage: Encryption

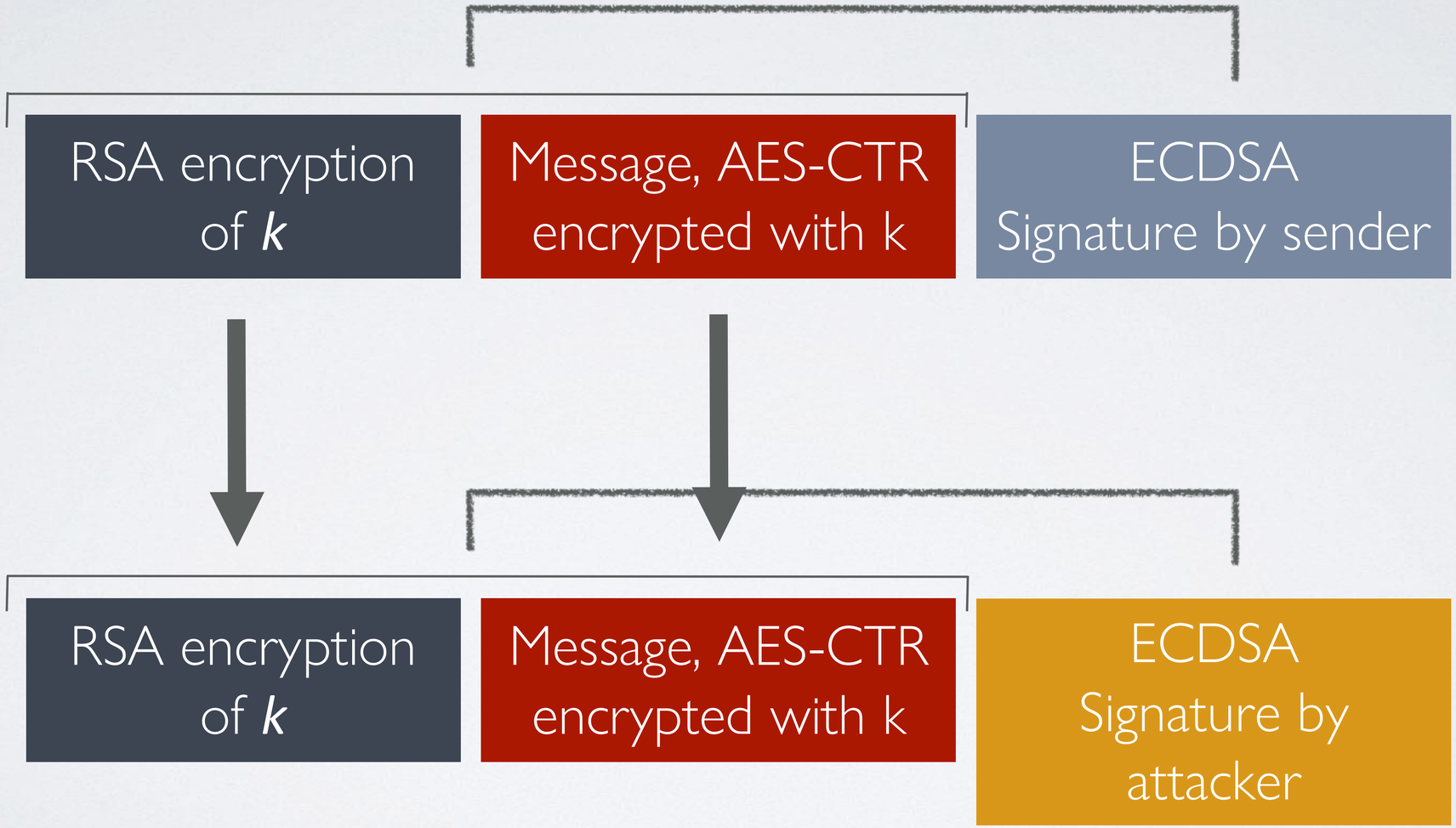
When a user turns on iMessage on a device, the device generates two pairs of keys for use with the service: an RSA 1280-bit key for encryption and an ECDSA 256-bit key on the NIST P-256 curve for signing. The private keys for both key pairs are saved in the device's keychain and the public keys are sent to Apple's directory service (IDS), where they are associated with the user's phone number or email address, along with the device's APNs address.

The user's outgoing message is individually encrypted for each of the receiver's devices. The public RSA encryption keys of the receiving devices are retrieved from IDS. For each receiving device, the sending device generates a random 128-bit key and encrypts the message with it using AES in CTR mode. This per-message AES key is encrypted using RSA-OAEP to the public key of the receiving device. The combination of the encrypted message text and the encrypted message key is then hashed with SHA-1, and the hash is signed with ECDSA using the sending device's private signing key. The

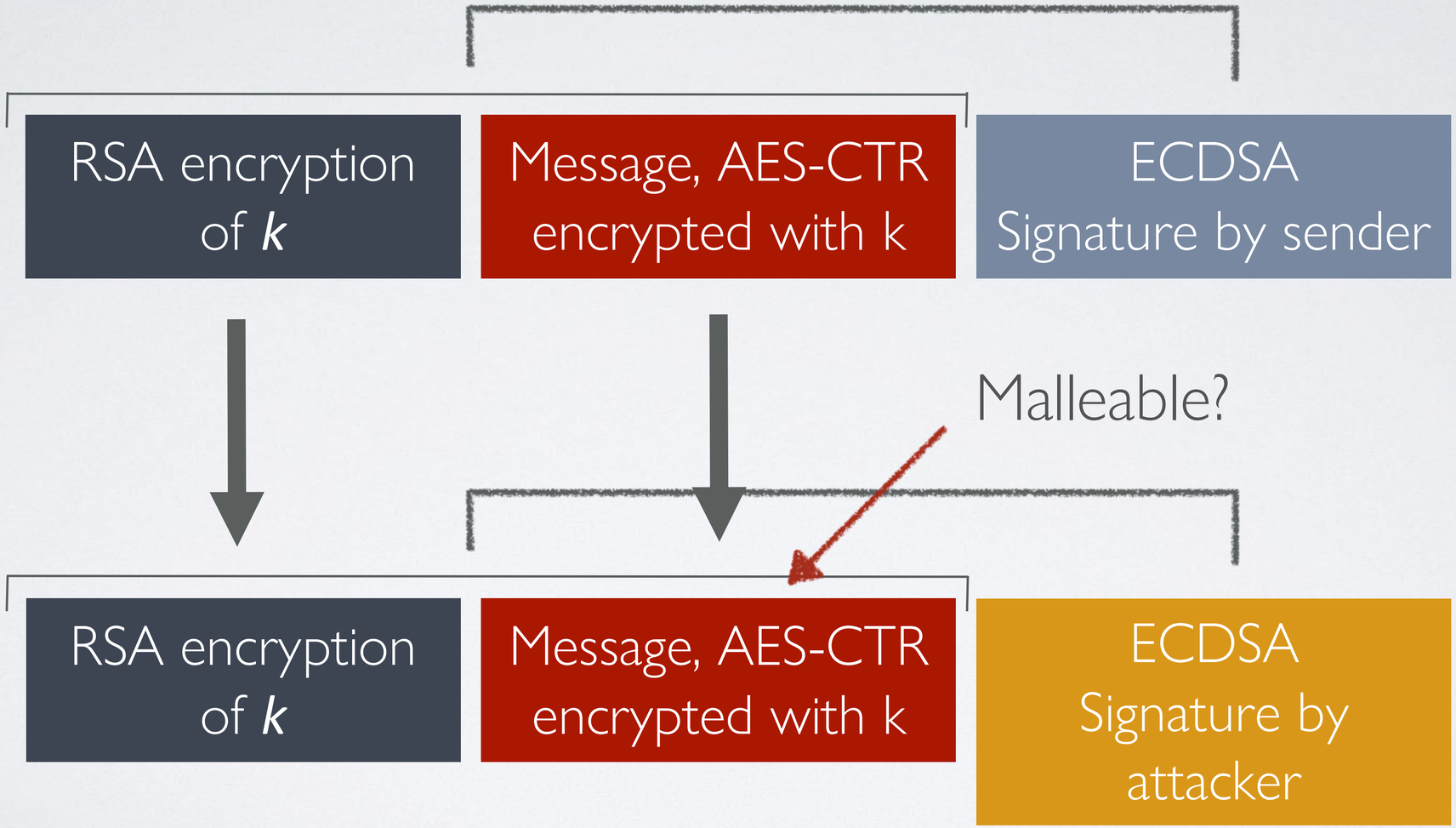
iMessage: Encryption



iMessage: Encryption



iMessage: Encryption



iMessage: the oracle attack

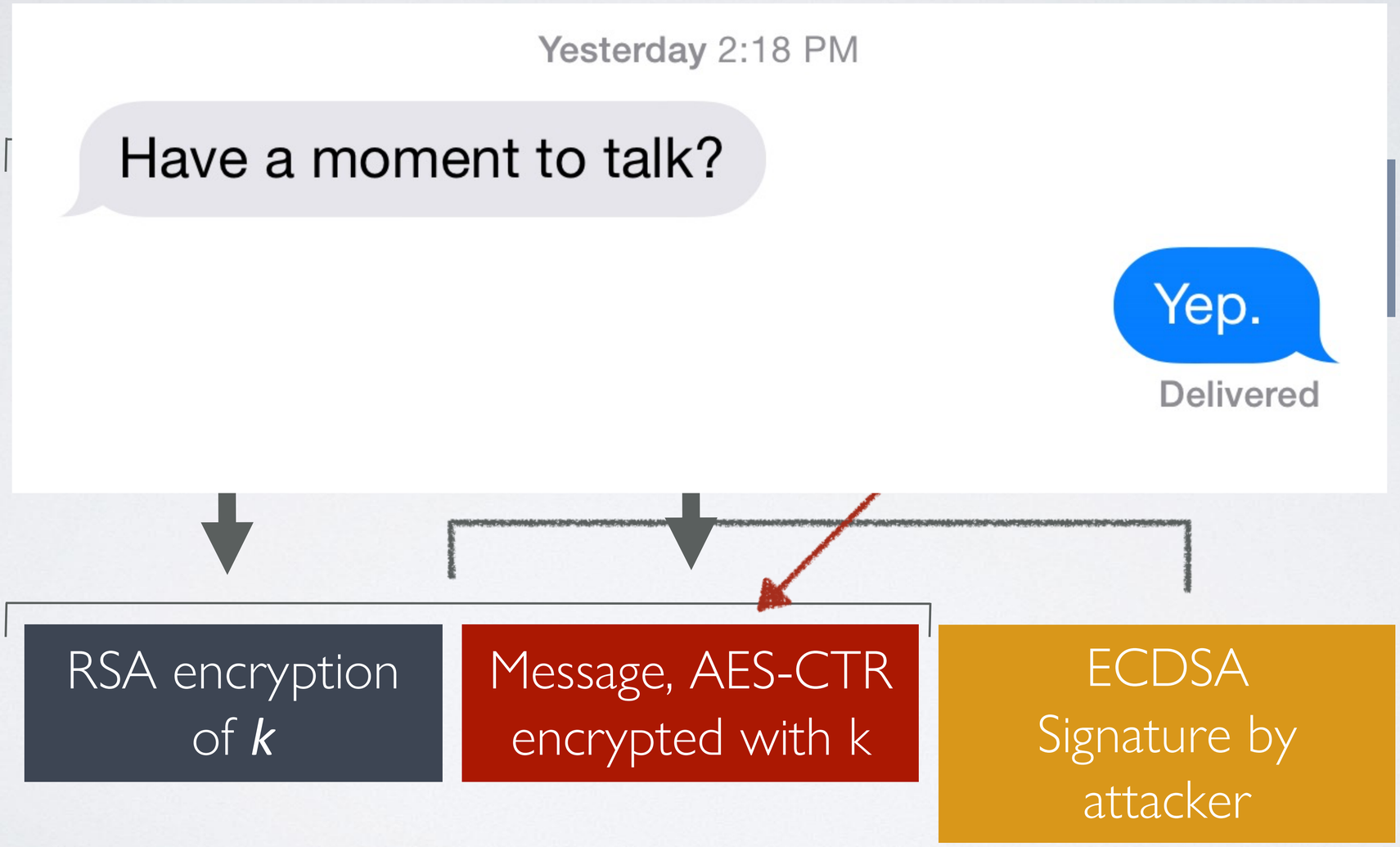
The old construction
Malleable ciphertexts and observable processing behavior created an oracle.

CTR needs no block padding
This was not a CBC padding-oracle bug in CTR mode.

The concrete result (2016)
A chosen-ciphertext attack exploited Huffman-compressed message data.

The outcome
Apple deployed mitigations. The protocol later changed substantially.

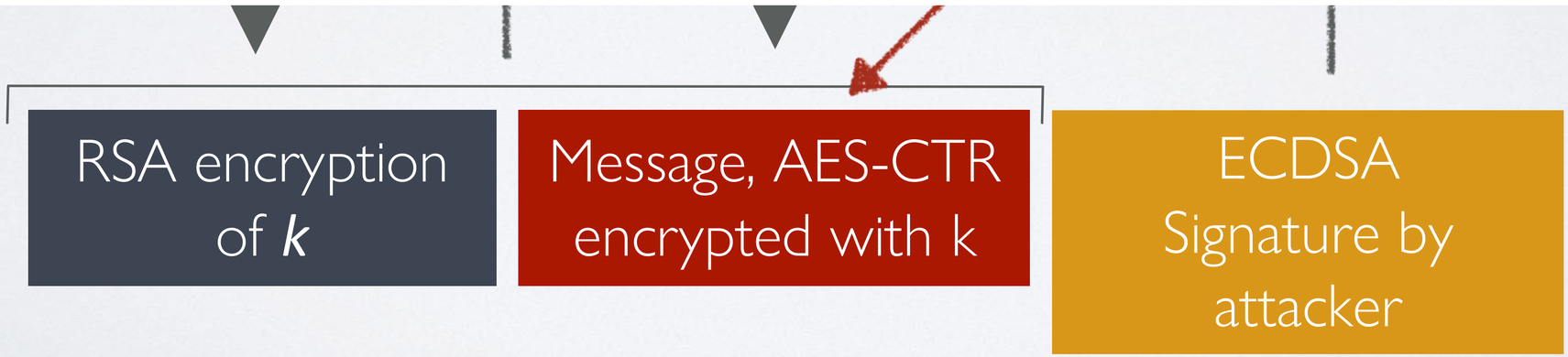
iMessage: Encryption



iMessage: Encryption

Here is an example of such a bplist:

```
D:   True
E:   'pair'
P:   <variable length binary data> (iMessage payload, deflate compressed)
U:   <128bit binary data> (iMessage UID)
c:   100
i:   <32bit integer> (messageId, same as in PUSH header)
sP:  mailto:tim_c@icloud.com  (sender URI)
t:   <256bit binary data> (sender Push-Token)
tP:  mailto:mark_z@facebook.com (receiver URI)
ua:  [Mac OS X,10.8.5,12F37,MacBookPro10,2] (sender OS and hardware version)
v:   1
```



iMessage after the case study

2016
Apple patched the chosen-ciphertext weaknesses in the historical construction.

2019
iMessage moved from RSA to elliptic-curve key establishment.

2024: PQ3
Hybrid post-quantum establishment and ongoing ratcheting strengthen message-key security.

The protocol lesson
Key establishment, authentication, and recovery after compromise are different goals.

The earlier RSA diagrams do not describe PQ3.

Conclusion

- Cryptography is challenging!
  - We fail to push best practices down to the engineering community
  - They fail to pull best practices from the literature, even years after vulnerabilities are known
  - Cryptosystems continue to become more complex and vulnerable
  - This process is not really tolerable anymore



Why these problems?

- Many problems result from TLS's use of "*pre-historic cryptography*" (- Eric Rescorla)
  - Export grade encryption
  - RSA-PKCS#1v1.5 encryption padding
  - RC4
  - DH parameter generation
  - Horrifying backwards compatibility requirements

Quite a bit

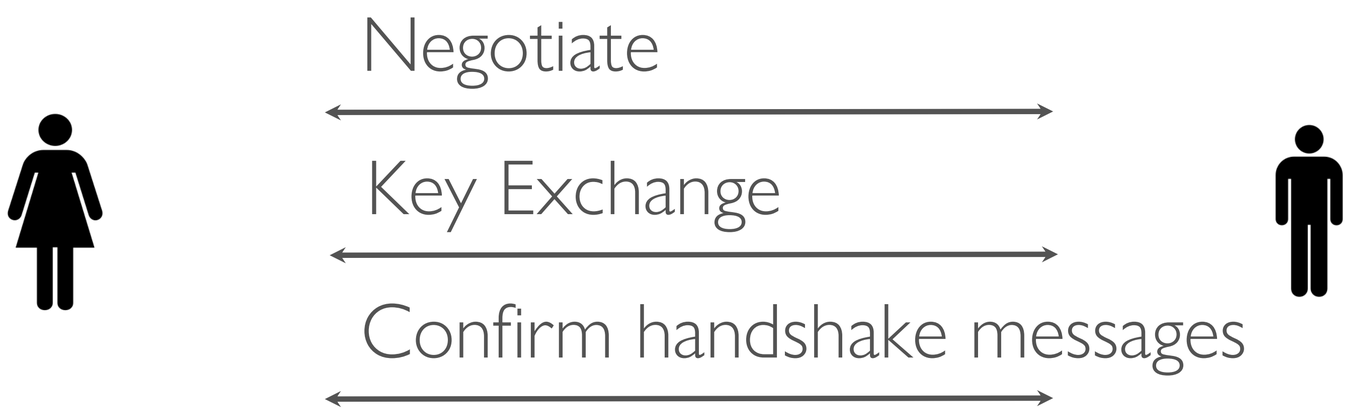
- Many problems result from TLS's use of "*pre-historic cryptography*" (- Eric Rescorla)
  - **Export**
  - RSA-P
  - RC4
  - DH par
  - Horrifyi

1995-~2000 (and onward)

Weakened "ciphersuites" with limited security
(e.g., 512-bit DH/RSA, 40-bit RC4)

TLS Negotiation

In TLS 1.2 and earlier, cipher-suite negotiation selects the key exchange and record-protection algorithms.



SSL/TLS

Bank of America

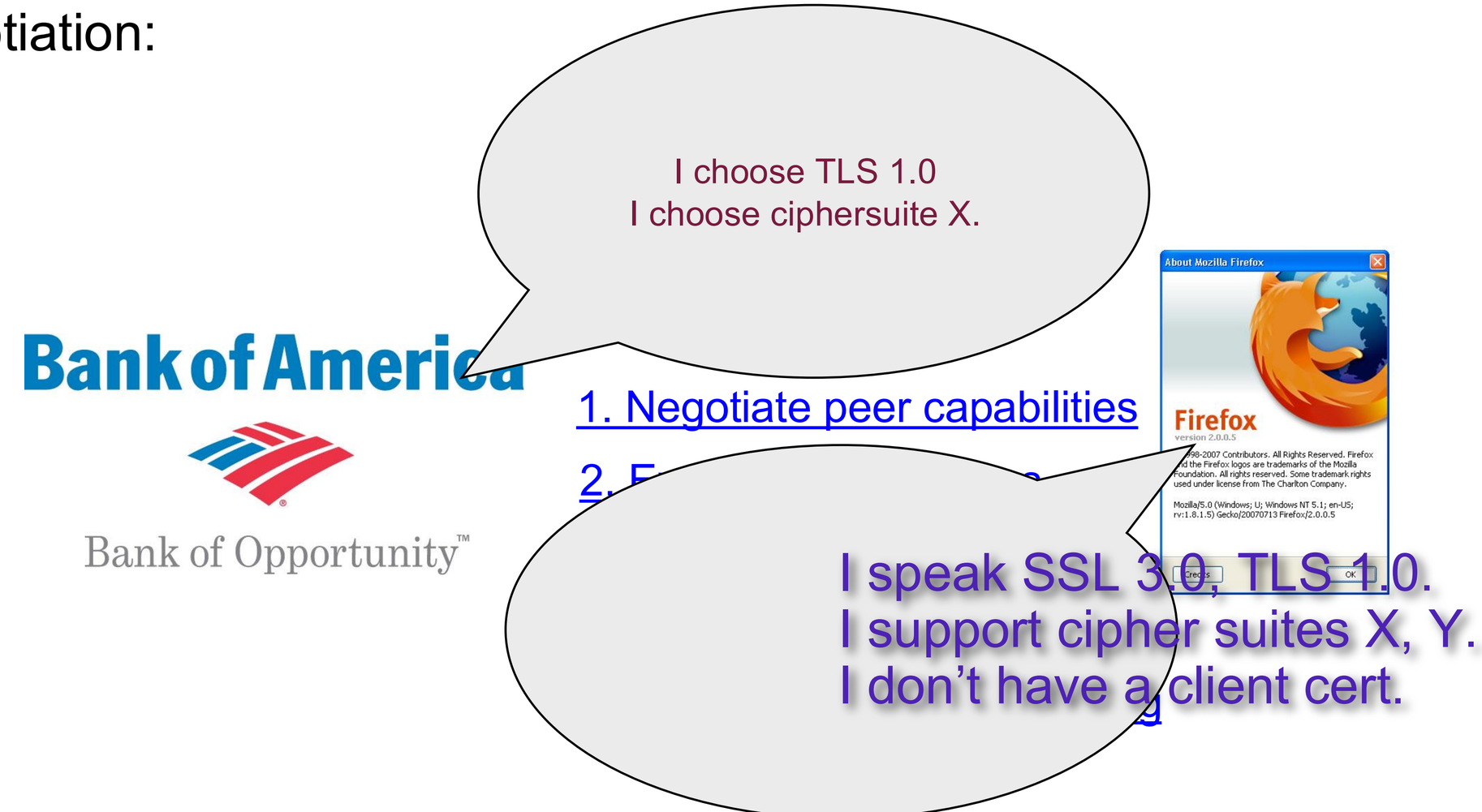
Bank of Opportunity™

1. Negotiate peer capabilities
2. Exchange certificates
3. Session key establishment
4. Secure communications
5. Session expiration/rekeying



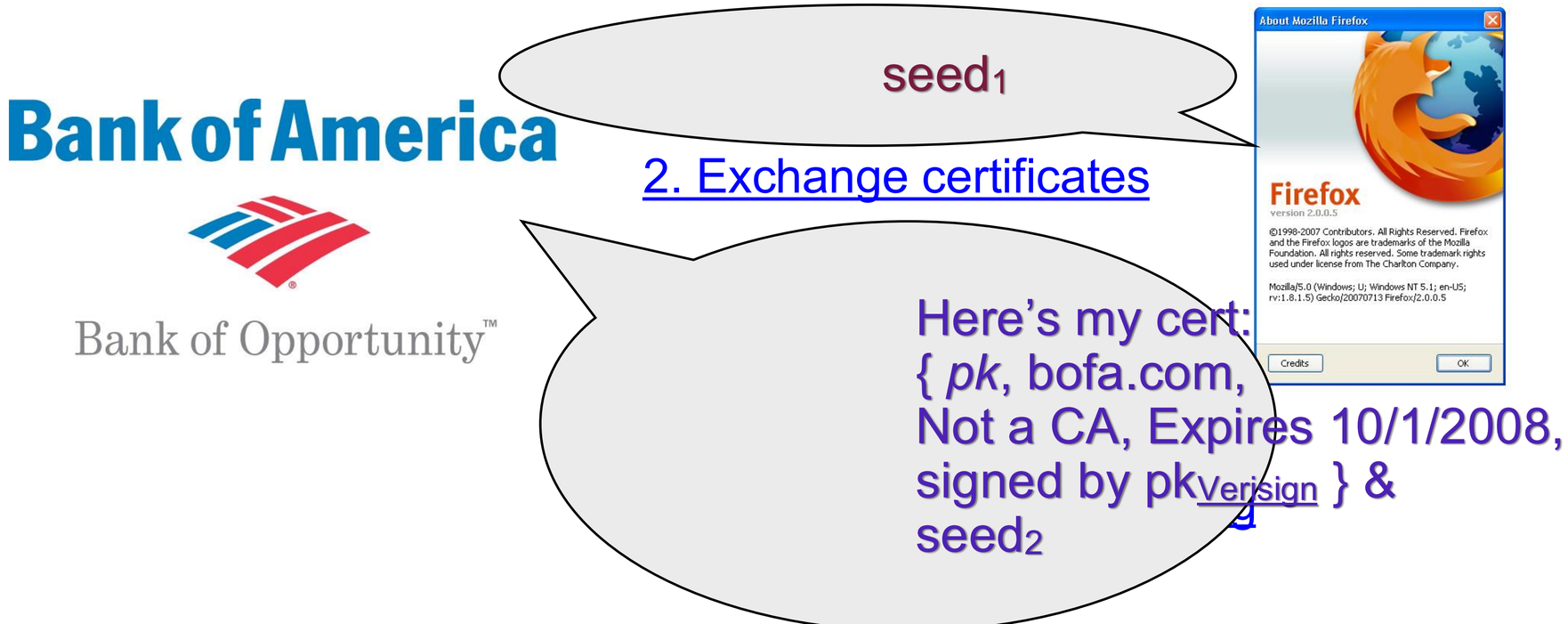
SSL/TLS

- Negotiation:



SSL/TLS

- Certificate Exchange



SSL/TLS

- Session key establishment
  - Various options
  - Legacy approach: RSA key transport



SSL/TLS

- Secure communication
  - Legacy records use separate MAC & encryption keys

Bank of America

Bank of Opportunity™

Communicate under k_s

1. Negotiate peer capabilities
2. Exchange certificates
3. Session key establishment
4. Secure communications
5. Session expiration/rekeying



SSL/TLS

- Key expiration/rekeying
  - Key has a defined lifetime
  - If session drops within that lifetime, we restart:

- This shortcut saves PK operations

Bank of America

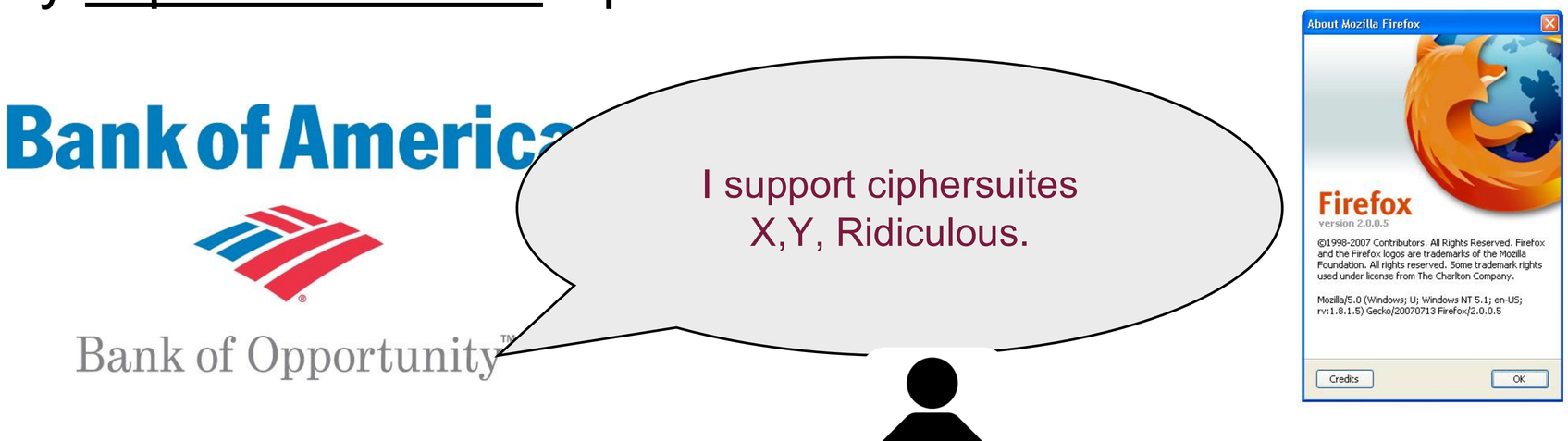
Bank of Opportunity™

1. Negotiate peer capabilities
2. Exchange certificates
3. Session key establishment
4. Secure communications
5. Session expiration/rekeying



Attacks on SSL2

- Many and varied...
- Major vulnerability:
  - Ciphersuite list not authenticated
  - Active attacker could modify the message to specify export-weakened ciphers



SSL3

- All of the problems with SSL2 fixed!
- Well, not quite:
  - Ciphersuite rollback attack (weaker)
  - Key-exchange algorithm rollback
  - Version rollback
  - (Weak) traffic analysis
  - Also, uses some non-standard primitives

CCS Rollback

- Most messages sent during client/server handshake are authenticated
  - Final MAC is sent at finish message
  - However, [change cipher spec] message is not included in the MAC
  - Tells the other party to start using encryption/authentication
  - Attacker can modify/drop this message!

CCS Rollback

- Normal protocol:

$$
\begin{array}{lll}
\ldots & & \\
1. & C \rightarrow S: & [\textbf{change cipher spec}] \\
2. & C \rightarrow S: & [\textbf{finished:}]\,\{a\}_k \\
3. & S \rightarrow C: & [\textbf{change cipher spec}] \\
4. & S \rightarrow C: & [\textbf{finished:}]\,\{a\}_k \\
5. & C \rightarrow S: & \{m\}_k \\
\ldots & &
\end{array}
$$

CCS Rollback

- MITM attack:

$$
\begin{array}{lll}
\ldots & & \\
1. & C \to M: & [\textbf{change cipher spec}] \\
2. & C \to M: & [\textbf{finished:}]\, \{a\}_k \\
2'. & M \to S: & [\textbf{finished:}]\, a \\
3. & S \to M: & [\textbf{change cipher spec}] \\
4. & S \to M: & [\textbf{finished:}]\, \{a\}_k \\
4'. & M \to C: & [\textbf{finished:}]\, a \\
5. & C \to M: & \{m\}_k \\
5'. & M \to S: & m \\
\ldots & &
\end{array}
$$

Key-Exchange Rollback

- SSL3 standard supports two ephemeral key exchange modes:
  - 1. Server publishes ephemeral RSA parameters (signed under its certified signing key)
  - 2. Server publishes ephemeral DH parameters
  - Client may be able to pick which to use
- Why ephemeral key exchange?
  - Advantages of Diffie-Hellman? RSA?

Key Exchange Rollback

Bank of America

Bank of Opportunity™





Normal RSA parameters: (N,e)

I assume *p* is the RSA modulus, and *g* is the RSA exponent. I ignore the extra value.

Since *p* is a prime, we can compute inverses. Recover k.

Version Rollback

- Release of SSL3 didn’t make SSL2 browsers go away
  - Servers still accepted SSL2 requests
  - Attacker could modify [client hello] message to specify SSL2
  - Server continues with SSL2 connection, attacker uses SSL2 attacks

Fallback can defeat security

The historical example
Legacy VPN clients could retry with a weaker protocol after a failure.

The general failure
An attacker induces failure, then benefits from the weaker fallback.

Modern policy
Reject insecure protocols even when a connection cannot otherwise complete.

Deployment example
New Windows Server 2025 RRAS setups disable PPTP/L2TP acceptance by default.

Traffic Analysis: SSL3

- Example:
  - First HTTP request typically looks like:

```
GET / HTTP/1.1
Host: cnn.com

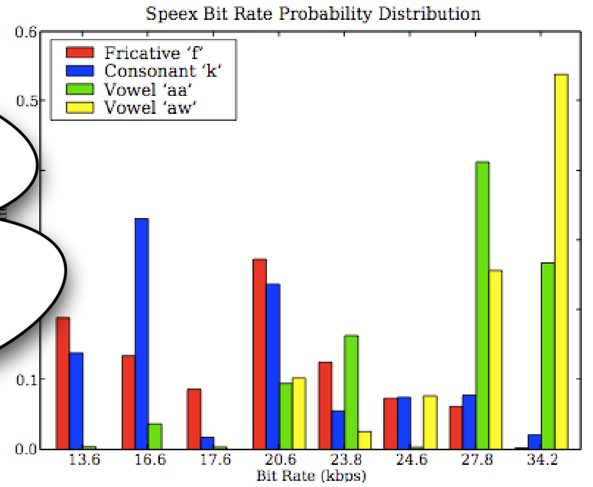
User-Agent: Mozilla/5.0 (Macintosh; U; Intel
Mac OS X 10_5_6; en-us) AppleWebKit/525.27.1
(KHTML, like Gecko) Version/3.2.1
Safari/525.27.1
```

  - From ciphertext length, we may be able to work out URL information

Traffic Analysis++

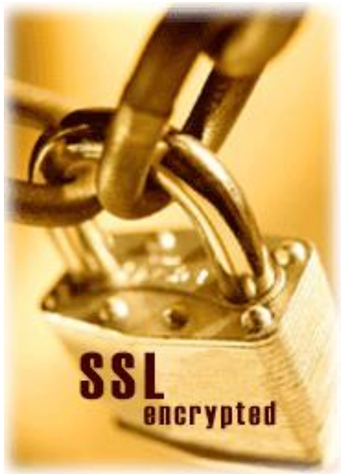
- Digression: The case of encrypted VoIP
  - Some VoIP protocols use VBR encoding, size of data packets depends on signal
  - Also include “silence suppression” (VAD)
  - Therefore, total traffic is [illegible] related to the contents of

Good news:
Most VoIP implementations don't actually use VBR/ supression



SSL stripping (2009)

- Moxie Marlinspike: SSLStrip
- Does not break SSL
- Instead: takes advantage of the way SSL is used



HTTP->HTTPS

- Banking example (2009):
  - SSL URLs begin with https://
  - But users rarely type the prefix



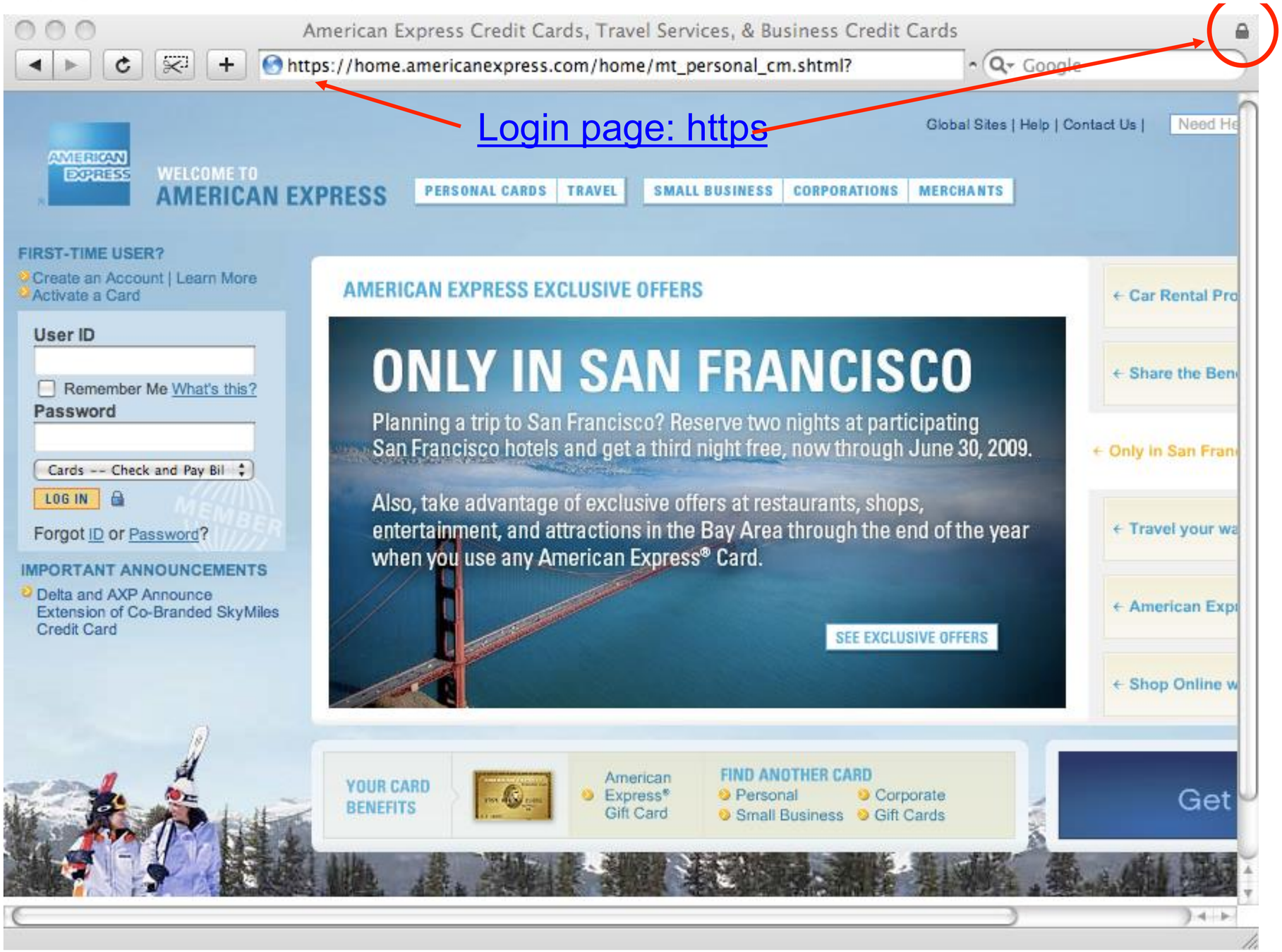
http://americanexpress.com

https://americanexpress.co

https://americanexpress.co

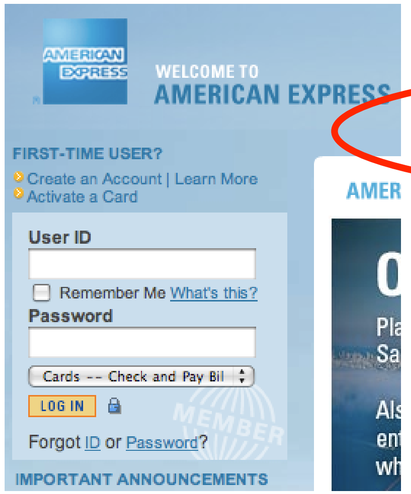


Historical case study: SSL stripping and HTTP fallback (2009)



HTTP->HTTPS

- If you can intercept the user's connection:
  - Don't redirect, or:
  - Redirect to malicious site, unsecured (http)



http://amex.com

http://secure.amex.com

https://secure.amex.com



HTTP->HTTPS

- If you can intercept the user's connection:
  - Homograph site: paypaI.com (with a capital i), or:
  - Use clever IDN tricks e.g.,

    https://www.gmaiI.com/accounts/ServiceLogin!f.ijjk.cn





HTTP->HTTP->HTTPS

- It can be worse:
  - Some sites served an HTTP page whose form submitted via HTTPS

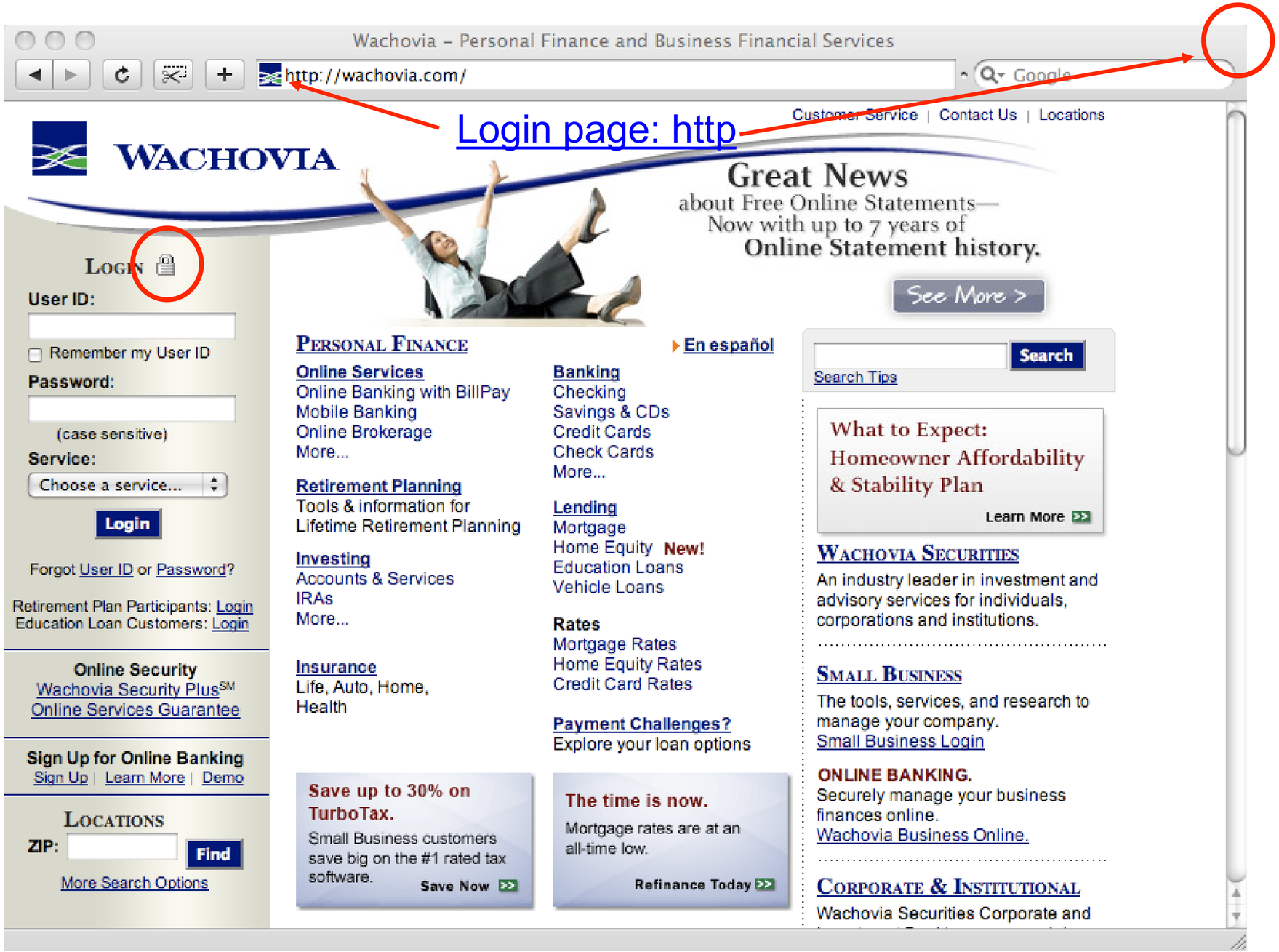


http://americanexpress.com





Historical case study: SSL stripping and HTTP fallback (2009)



SSL stripping: defenses today

HSTS
A remembered HTTPS-only policy prevents a browser from falling back to HTTP.

Preloading
A built-in policy can protect even the first visit to a listed site.

HTTPS-first browsing
Browsers try HTTPS automatically. Warnings and fallback behavior depend on policy.

The remaining boundary
An unprotected first visit or permitted HTTP fallback can still expose the user.

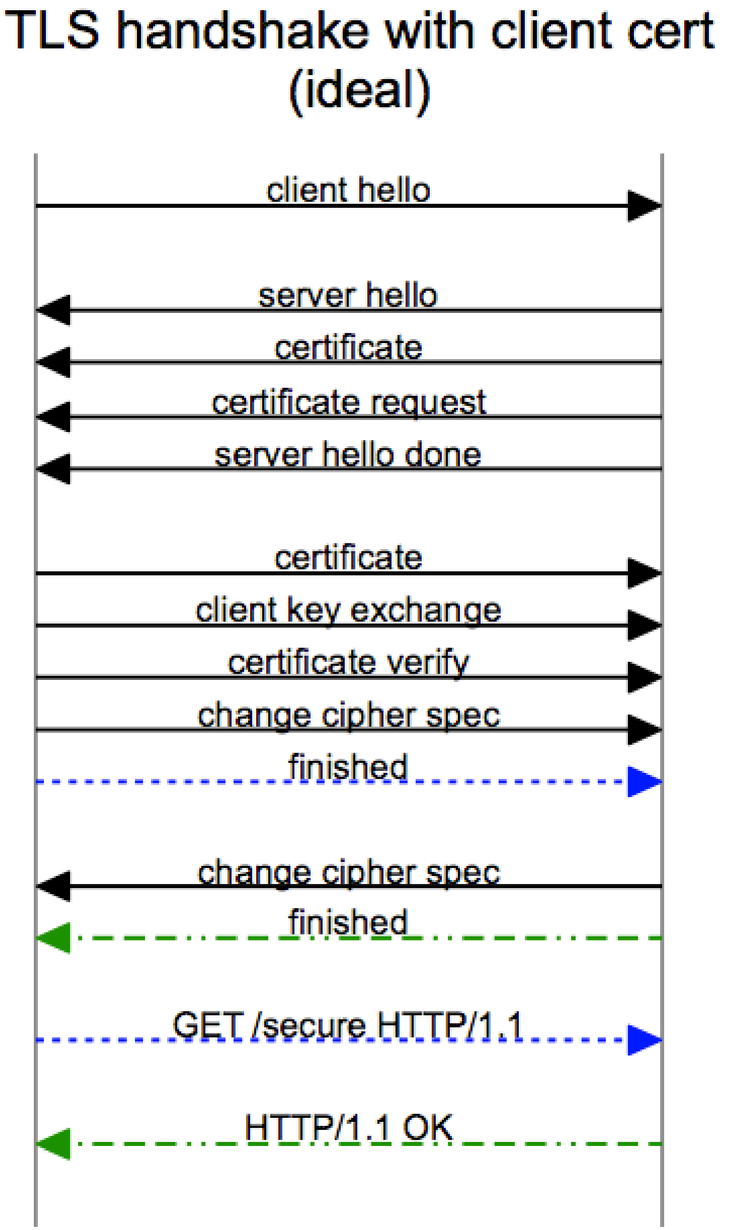
The old banking screenshots illustrate the attack, not current site behavior.

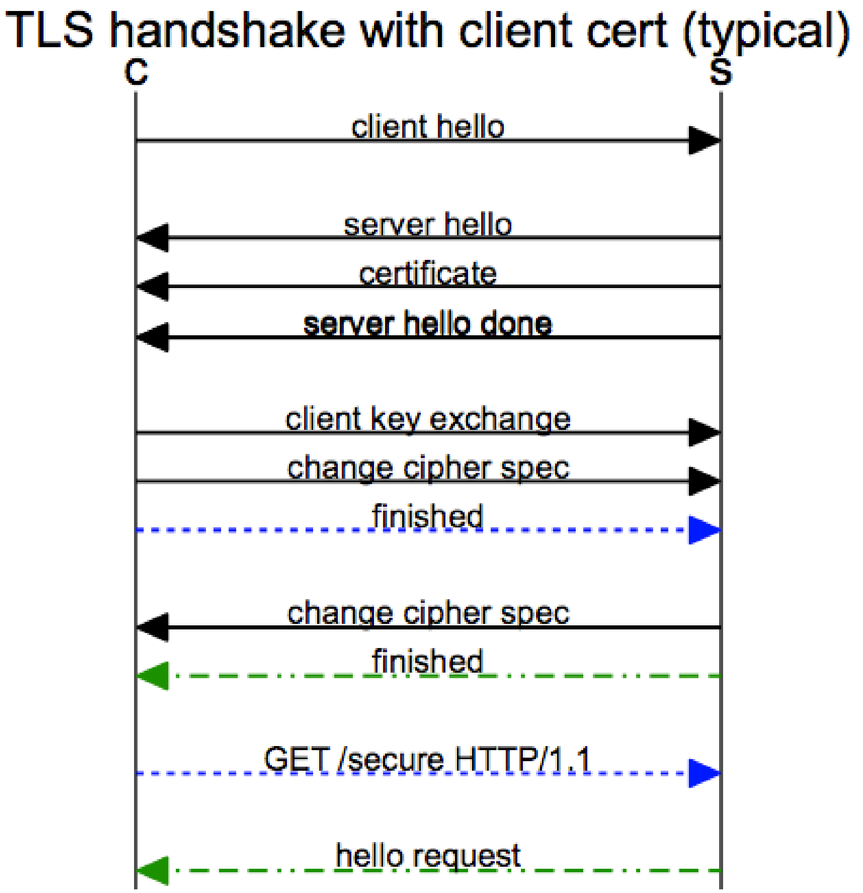
Injecting Prefixes

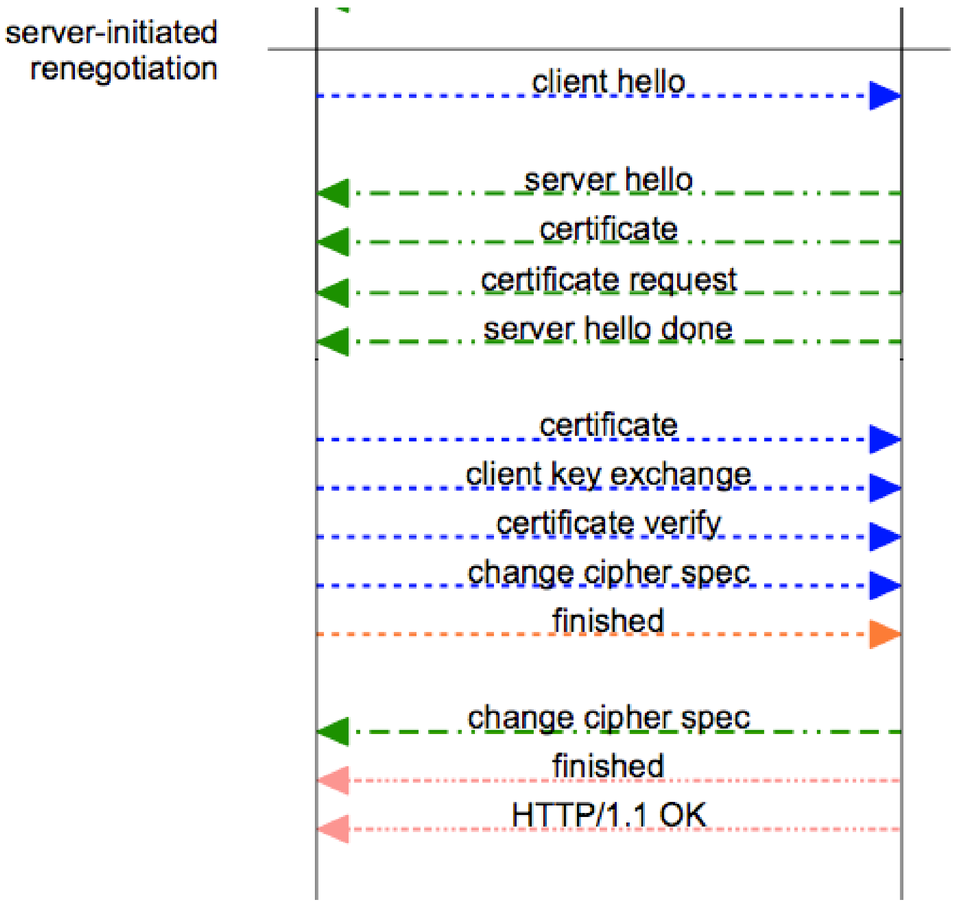
- Ray, Dispensa, 11/2009:
  - Many web servers require client-side auth, but only for certain resources

```
GET /highsecurity/index.html HTTP/1.1
Host: example.com
Connection: keep-alive
```

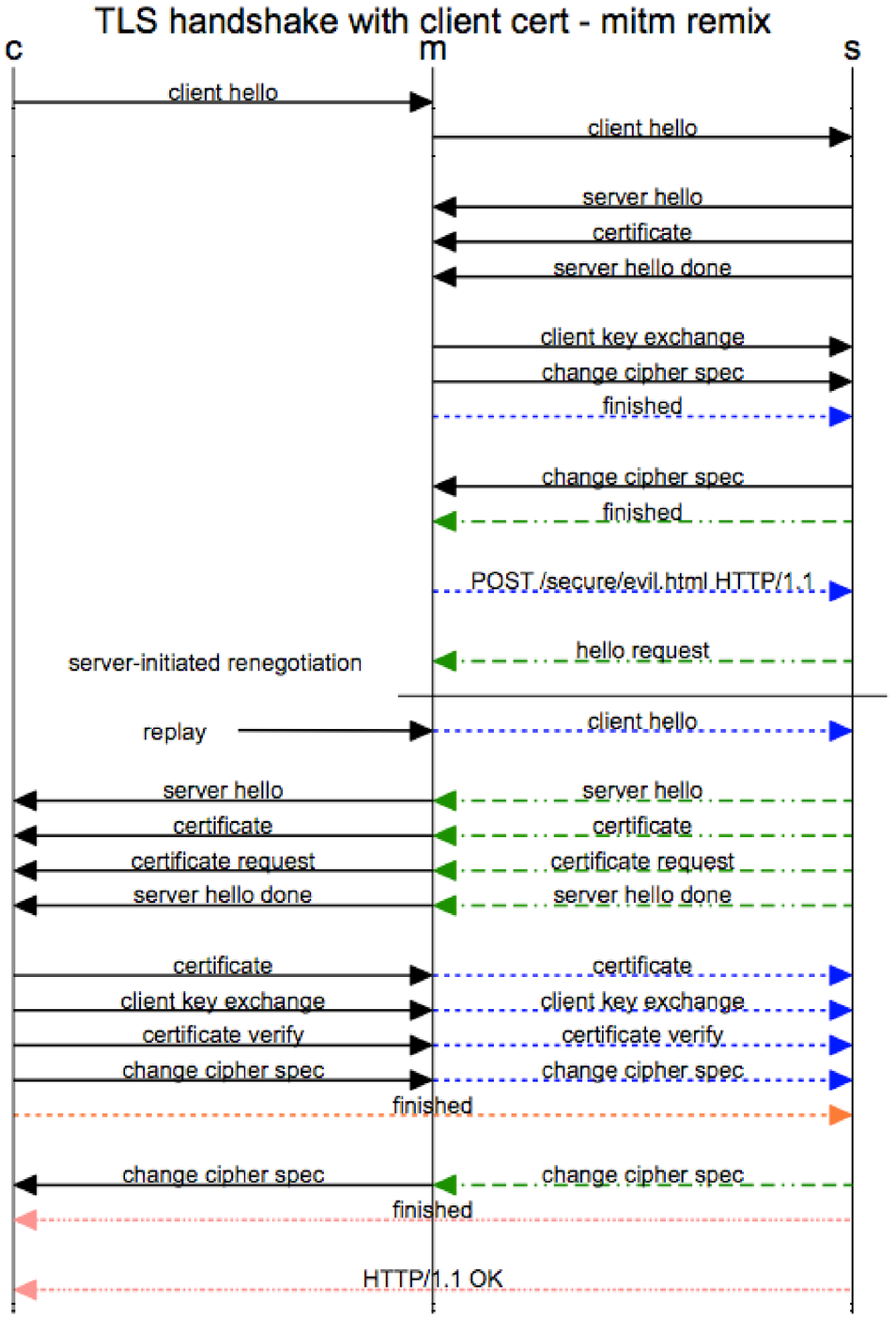
  - This may require an on-the-fly TLS re-negotiation


TLS handshake with client cert (ideal)
client hello
server hello
certificate
certificate request
server hello done
certificate
client key exchange
certificate verify
change cipher spec
finished
change cipher spec
finished
GET /secure HTTP/1.1
HTTP/1.1 OK


TLS handshake with client cert (typical)
C
S
client hello
server hello
certificate
server hello done
client key exchange
change cipher spec
finished
change cipher spec
finished
GET /secure HTTP/1.1
hello request


server-initiated
renegotiation
client hello
server hello
certificate
certificate request
server hello done
certificate
client key exchange
certificate verify
change cipher spec
finished
change cipher spec
finished
HTTP/1.1 OK

Historical case study: TLS renegotiation attack (2009)



DECT

- Digital Enhanced Cordless Telephone protocol
  - European standard, now in US
  - Interoperable devices
  - Connects base station (FT) to handsets (PT)
- Tools:
  - DECT Standard Cipher (DSC)
  - DECT Standard Authentication Algorithm (DSAA)



DECT

- Step 1: Pairing
  - User enters a 4-digit PIN into handset and base
  - Base generates a 64-bit seed, combined with PIN to generate shared key (UAK)
  - Base and handset conduct challenge/re

Total entropy of UAK:
77.288 bits (64-bit seed + PIN)
Much less if PRNG is bad!



DECT

- Step 2: Authentication
  - Two 

commended one:



In common mode, only the handset is authenticated!

DECT Attack

- Step 2: Authentication
  - Two 

commended one:



de?

DECT: what changed

The legacy attacks
The preceding examples concern the original DSAA/DSC generation.

Stronger algorithms
DECT added AES-128-based DSAA2 and DSC2 in 2011–2012.

Standards guidance
ETSI recommended these replacements for new products.

Deployment still matters
Check the device and negotiated mode. The standard alone does not ensure use.

Example: DTCP

- BluRay & HD-DVD Disks
  - Contains “protected” area that can’t be read using normal Drive protocol
  - Embeds secret “Binding Nonce”



DTCP Protocol

- Digital Transmission Content Protection
  - Runs between Drive and Host
  - Encrypts & Authenticates Communications





DTCP

- One layer of protection for HD-DVD/BluRay
  - Encrypts/authenticates content traversing unprotected bus lines



Attacker







http://nanocr.eu/picture.php

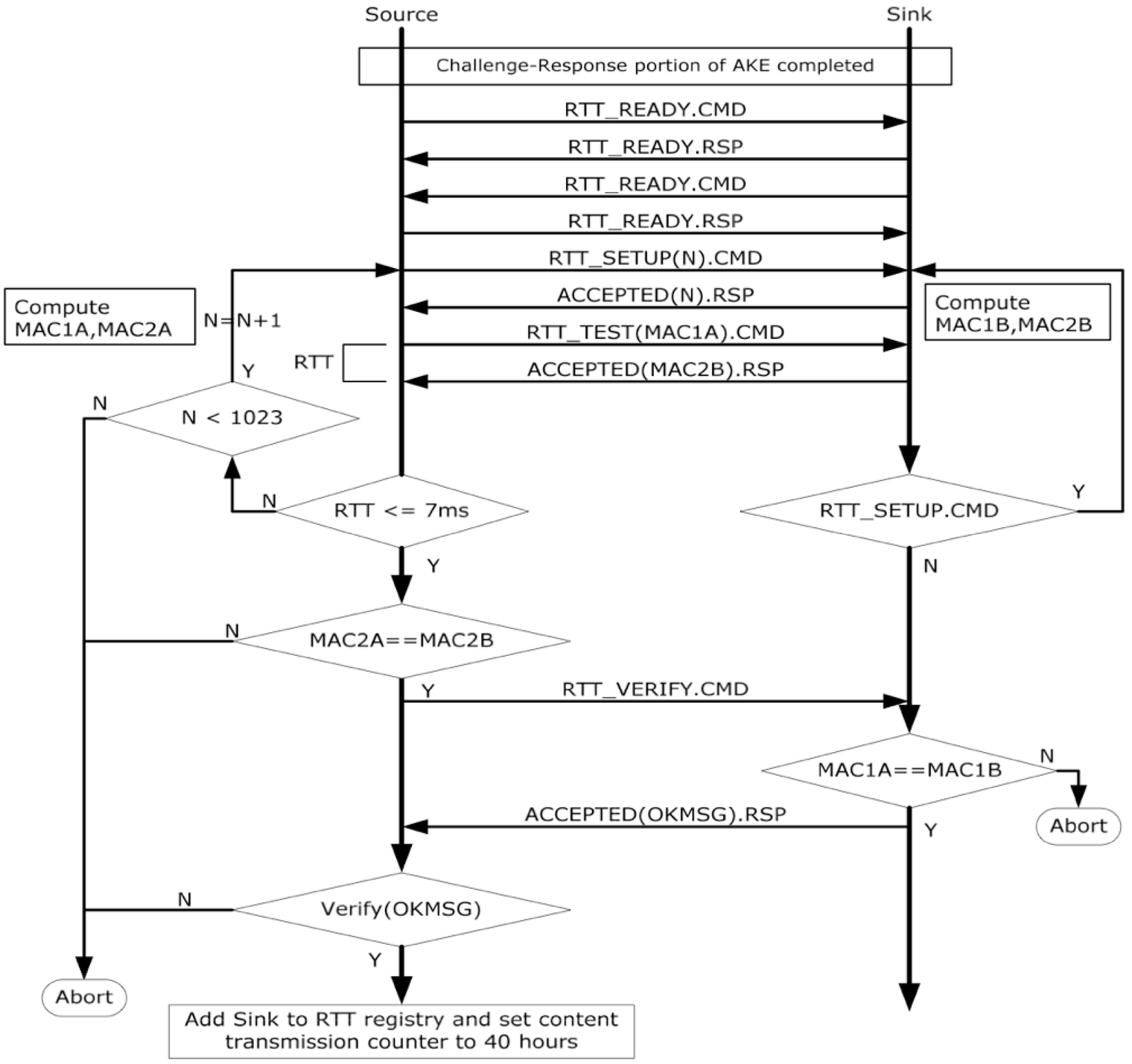
DTCP AKE

- Authenticated Key Exchange
  - EC Diffie-Hellman Protocol
  - Each device has a certificate & secret key
  - Devices also have a certificate revocation list, to prevent communication with hacked devices

DTCP AKE (v1.4)






Source
Sink
Challenge-Response portion of AKE completed
RTT_READY.CMD
RTT_READY.RSP
RTT_READY.CMD
RTT_READY.RSP
RTT_SETUP(N).CMD
ACCEPTED(N).RSP
RTT_TEST(MAC1A).CMD
ACCEPTED(MAC2B).RSP
RTT_VERIFY.CMD
ACCEPTED(OKMSG).RSP
Compute MAC1A,MAC2A
N=N+1
RTT
Compute MAC1B,MAC2B
N < 1023
RTT <= 7ms
RTT_SETUP.CMD
MAC2A==MAC2B
MAC1A==MAC1B
Verify(OKMSG)
Abort
Abort
Add Sink to RTT registry and set content transmission counter to 40 hours
Y
N

Other Attacks

- Replay Attacks
  - Attacker replays older messages
  - Can be countered with timestamps, nonces and sequence counters
- Cut & Paste
  - Malleable encryption scheme like CBC
  - Can be countered with MACs
- Reflection
  - If party A sends a message, just bounce it back

Discussion

- We’ve seen standards with problems
  - Usually the cryptanalysis comes after the standard is released, and products in the field
  - Why?

Next Time

- Next lecture:
  - How do we design secure protocols?

END

Ciphersuite Rollback

Bank of America

Bank of Opportunity™



Ciphersuite Rollback

Bank of America

Bank of Opportunity™





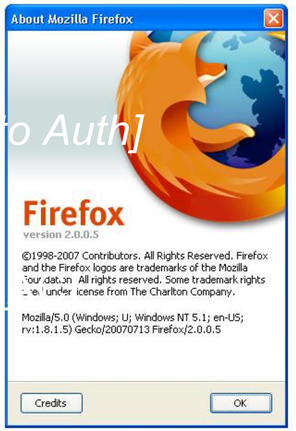
Ciphersuite Rollback

- Big caveat:
  - Only works when client asks for authentication <u>without</u> encryption



Bank of America

Bank of Opportunity™



Server thinks encryption is disabled, but gets an encrypted MAC